CNN: Dog/Cat và MNIST

Notebook tổng hợp hai bài toán phân loại ảnh bằng NumPy from scratch, TensorFlow/Keras và PyTorch.

Phần A — Phân loại ảnh Dog/Cat

Phân loại chó và mèo bằng CNN

2. Kiến thức nền: CNN và các mô hình cải tiến

2.1 Bài toán

Ảnh là một tensor có dạng `(chiều cao, chiều rộng, số kênh)`. Ảnh RGB có 3 kênh; ảnh xám có 1 kênh. Bài toán nhị phân chó/mèo cần dự đoán một xác suất `p`: thường quy ước `p >= 0.5` là chó và ngược lại là mèo.

2.2 Fully-connected, CNN và kernel (giải thích trực quan)

Fully-connected (Dense) nghĩa là mọi neuron tầng trước nối với mọi neuron tầng sau. Một neuron tính `z=w1*x1+...+wn*xn+b`. MLP cơ bản chủ yếu gồm các Dense layer; CNN thường chỉ dùng Dense ở cuối.

Vì sao CNN thường đặt Dense ở cuối?

Convolution và Dense đảm nhiệm hai vai trò khác nhau:

`Ảnh → Conv/Pooling (trích xuất đặc trưng) → Dense (ra quyết định) → nhãn`
Các lớp Conv nhìn vùng cục bộ nhưng giữ feature map dạng `chiều cao × chiều rộng × số kênh`. Qua nhiều lớp, chúng phát hiện dần cạnh, góc, bộ phận và hình dạng. Khi đã có các đặc trưng này, Dense kết hợp **toàn bộ** chúng để trả lời "đây là chó hay mèo?". Nếu đưa Dense ngay sau pixel, phải flatten ảnh, làm mất quan hệ hàng–cột và tạo hàng triệu trọng số; vì vậy Dense thường để cuối, khi dữ liệu đã được nén thành đặc trưng hữu ích. Dense lúc này giống một bộ ra quyết định, không còn phải học lại cách nhận cạnh từ từng pixel.

Dense ở cuối không phải luật bắt buộc. Một thiết kế phổ biến là `Conv → Global Average Pooling → Output`: mỗi feature map được lấy trung bình thành một số, rồi lớp output nhỏ dự đoán lớp. Cách này giữ lợi ích của Conv nhưng có ít tham số và ít overfitting hơn `Flatten → Dense` lớn.

Ảnh là một dạng lưới: ảnh xám 4×4 là ma trận pixel hàng–cột; ảnh RGB có dạng `(H,W,3)`. Pixel kề nhau thường thuộc cùng cạnh/đường nét. CNN nhận tensor này trực tiếp nên giữ quan hệ hàng xóm; Dense phải flatten thành vector và làm mất thông tin đó. Ảnh RGB 224×224 có 150.528 giá trị; nối với 1.000 neuron cần 150.528.000 trọng số.

Kernel/filter là ma trận nhỏ (thường 3×3) gồm các trọng số. Nó nhìn một vùng 3×3, nhân từng ô rồi cộng thành một số; sau đó trượt sang vùng kế tiếp. Cùng một bộ 9 trọng số được dùng lại ở mọi vị trí (weight sharing), nên phát hiện một mẫu dù mẫu nằm bên trái hay bên phải.

Ví dụ một kernel phát hiện **cạnh dọc**:

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Vùng ảnh đang xét:

$$P = \begin{bmatrix} 0 & 0 & 255 \\ 0 & 0 & 255 \\ 0 & 0 & 255 \end{bmatrix}$$

Nhân từng ô tương ứng rồi cộng:

```
(-1×0) + (0×0) + (1×255)
+(-1×0) + (0×0) + (1×255)
+(-1×0) + (0×0) + (1×255)
= 0 + 255 + 0 + 255 + 0 + 255 = 765
```

`765` lớn nghĩa là mẫu "trái tối → phải sáng" khớp mạnh; feature map sáng tại vị trí này. Với vùng đồng nhất:

$$P = \begin{bmatrix} 100 & 100 & 100 \\ 100 & 100 & 100 \\ 100 & 100 & 100 \end{bmatrix}$$

ta được `(-100+0+100) × 3 = 0`, nên không có cạnh dọc. Sáng trái/tối phải cho kết quả âm (ReLU thường đưa về 0); kernel khác có thể học hướng ngược lại. Kernel xoay 90° nhạy cạnh ngang; trọng số theo đường chéo nhạy cạnh chéo. Sobel/Prewitt là kernel thiết kế sẵn; CNN thường học kernel từ dữ liệu.

Ban đầu kernel học được khởi tạo ngẫu nhiên nhỏ (He/Xavier), chưa biết cạnh là gì. Sau mỗi batch, backpropagation tính đạo hàm của loss theo từng trọng số; optimizer cập nhật `w_new=w_old−learning_rate×gradient`. Nếu phát hiện cạnh giúp dự đoán đúng hơn, các trọng số sẽ dần điều chỉnh theo hướng làm phản hồi cạnh hữu ích hơn.

Nói "cục bộ → phức tạp" là nói về **receptive field**: kernel đầu chỉ nhìn 3×3 nên học mẫu cục bộ; lớp sau nhận nhiều feature map nên có thể ghép cạnh thành góc/đường cong; lớp sâu nhìn vùng rộng hơn và thường biểu diễn bộ phận như mắt/tai rồi hình dạng toàn vật. Đây là inductive bias do kết nối cục bộ và xếp chồng lớp, một xu hướng được quan sát qua feature map, không phải quy tắc gán nhãn cứng cho mọi mạng.

2.3 Phép tích chập: ví dụ từng bước

Ta dùng ảnh nhỏ 4×4 và kernel 2×2 để nhìn được toàn bộ phép tính:

```
Ảnh A = ⎡ 1  2  3  0 ⎤        Kernel K = ⎡ 1  0 ⎤
        ⎢ 4  5  6  1 ⎥                   ⎣ 0  1 ⎦
        ⎢ 7  8  9  2 ⎥
        ⎣ 0  1  2  3 ⎦
```

Thiết lập `stride = 1` (mỗi lần dịch 1 ô) và `padding = 0` (không thêm viền). Kernel bắt đầu ở góc trên trái, phủ lên cửa sổ:

```
Cửa sổ 1 = ⎡ 1  2 ⎤        Tính: 1×1 + 2×0 + 4×0 + 5×1 = 6
           ⎣ 4  5 ⎦
```

Trượt kernel sang phải **một cột**:

```
Cửa sổ 2 = ⎡ 2  3 ⎤        Tính: 2×1 + 3×0 + 5×0 + 6×1 = 8
           ⎣ 5  6 ⎦
```

Trượt tiếp sang phải:

```
Cửa sổ 3 = ⎡ 3  0 ⎤        Tính: 3×1 + 0×0 + 6×0 + 1×1 = 4
           ⎣ 6  1 ⎦
```

Khi hết cột, kernel quay về cột đầu và trượt xuống một hàng. Lặp lại cho mọi vị trí, ta thu được feature map:

```
Feature map = ⎡  6   8   4 ⎤
              ⎢ 12  14   8 ⎥
              ⎣  7  11  12 ⎦
```

Vì ảnh có kích thước 4×4, kernel 2×2 và stride 1 nên có `4−2+1=3` vị trí theo mỗi chiều → đầu ra 3×3. Nếu `stride=2`, kernel nhảy 2 ô nên có ít vị trí hơn. `padding='same'` thêm các ô 0 quanh biên để giữ kích thước gần bằng ảnh; `valid` không thêm biên. Công thức một chiều là `floor((n + 2p − k)/s) + 1`.

Từ kernel đến feature map, ReLU và pooling: toàn bộ chuỗi xử lý

Hãy coi một lớp CNN như một dây chuyền. Ảnh ban đầu là đầu vào; các bước sau không xuất hiện ngẫu nhiên mà được xếp để biến ảnh thành đặc trưng dễ phân loại:

`Ảnh (pixel) → Kernel → Convolution → Feature map → ReLU → Pooling`
Bước 1 – Kernel từ đâu ra? Kernel là ma trận trọng số nhỏ, ví dụ 3×3. Khi lập trình `Conv2D(filters=32, kernel_size=3)`, con người chỉ yêu cầu tạo 32 kernel kích thước 3×3. Các con số cụ thể thường khởi tạo ngẫu nhiên nhỏ; mạng sẽ học chúng qua loss và backpropagation. (Các kernel Sobel trong xử lý ảnh cổ điển là trường hợp khác: do con người thiết kế sẵn.)

Bước 2 – Convolution làm gì với kernel? Kernel trượt trên ảnh. Ở mỗi vị trí, lấy vùng ảnh cùng kích thước kernel, nhân từng ô tương ứng, cộng lại và cộng bias. Mỗi vị trí tạo **một số**. Quét hết ảnh tạo ma trận các số; ma trận đó gọi là feature map. Ví dụ ảnh 4×4 và kernel 2×2

với stride 1 tạo feature map 3×3 (`4−2+1=3` mỗi chiều). Feature map là bản đồ "kernel phản hồi mạnh ở đâu", không phải ảnh màu mới.

Bước 3 – Nhiều kernel để tìm nhiều mẫu: Một kernel chỉ phản hồi mạnh với một kiểu tương quan trọng số. Lớp có 32 kernel sẽ tạo 32 feature map (mỗi map có thể nhạy với một cạnh, texture hoặc mẫu khác). Chồng 32 map lại tạo tensor đặc trưng có dạng `H×W×32` .

Bước 4 – ReLU đặt ở đâu và làm gì? Sau convolution, ta thường áp dụng hàm kích hoạt ReLU cho từng ô của feature map: `ReLU(x)=max(0,x)` . Ví dụ `[-2, 3, -1, 5] → [0, 3, 0, 5]` . ReLU không phải kernel và không tự tìm cạnh; nó chỉ loại giá trị âm, giữ phản hồi dương, tạo tính phi tuyến để nhiều lớp có thể học quan hệ phức tạp. Chuỗi thường là `Conv → ReLU` .

Bước 5 – Pooling đặt ở đâu và làm gì? Sau ReLU, MaxPooling nhìn các cửa sổ nhỏ (thường 2×2), lấy giá trị lớn nhất. Ví dụ:

```
[1 5]  → 5       [2 0]
[3 4]            → 2
```

Hai cửa sổ trên được rút thành `[5, 2]` . Pooling làm feature map nhỏ hơn (giảm bộ nhớ và tính toán), giữ phản hồi nổi bật và giúp mô hình bớt nhạy với việc đặc trưng dịch vài pixel. Pooling không học kernel và không tạo mẫu mới; nó chỉ tổng hợp các giá trị lân cận.

Một khối CNN điển hình vì thế là:

```
Conv (nhiều kernel học được)
        ↓
ReLU (giữ phản hồi dương, tạo phi tuyến)
        ↓
MaxPooling (giảm kích thước, giữ phản hồi mạnh)
```

Lặp nhiều khối làm receptive field lớn dần; cuối mạng dùng Global Average Pooling hoặc Dense để biến các đặc trưng thành dự đoán chó/mèo.

Suy ra công thức kích thước đầu ra

Xét một chiều của ảnh. Gọi `n` là kích thước ảnh, `k` là kích thước kernel, `p` là số ô padding thêm ở **mỗi bên**, và `s` là stride.

Sau padding, chiều dài là `n+2p` . Kernel dài `k` , nên số khoảng trống mà kernel có thể bắt đầu trượt là `(n+2p)−k` . Vì mỗi lần nhảy `s` ô, số bước trượt hợp lệ là `floor(((n+2p)−k)/s)` . Có thêm vị trí bắt đầu đầu tiên (bước 0), nên phải cộng `1` :

```
output = floor((n + 2p − k) / s) + 1
```

Ví dụ `n=5, k=3, p=0, s=1` :

```
output = floor((5 + 0 − 3)/1) + 1 = 3
```

Kernel đặt được tại vị trí bắt đầu 0, 1, 2 — tổng cộng 3 vị trí. Nếu `s=2` , chỉ đặt tại 0 và 2, nên `floor((5−3)/2)+1=2` . Nếu thêm padding `p=1` , ảnh dài `5+2=7` , kernel 3 và stride 1

cho `floor((7−3)/1)+1=5`; đây là lý do `same` giữ kích thước trong trường hợp stride 1 và kernel 3. Công thức được áp dụng độc lập cho chiều cao và chiều rộng.

Convolution không "hiểu" như người; nó tạo số lớn tại nơi vùng ảnh khớp mẫu trọng số. Nhiều kernel tạo nhiều feature map; ReLU giữ tín hiệu dương và pooling giảm lưới.

2.4 Các lớp chính trong CNN – giải thích từng lớp

2.4.1 `Conv2D` – lớp tích chập

Đầu vào là tensor `(H,W,C_in)`. Lớp học `filters` kernel kích thước `k×k×C_in`; mỗi kernel quét qua mọi kênh, nhân–cộng và thêm bias để tạo một feature map. `filters=32` nghĩa là 32 kernel và 32 kênh đầu ra. `stride` là bước trượt, `padding` thêm viền; kích thước là `floor((n+2p-k)/s)+1`. Conv học mẫu cục bộ, chưa trực tiếp quyết định chó/mèo.

Ví dụ: đầu vào `32×32×3`, `filters=16`, `kernel_size=3`, `padding="same"` cho đầu ra `32×32×16`; số tham số `16×(3×3×3+1)=448`. Mỗi output pixel là tổng có trọng số trên một vùng ảnh, nên các vị trí lân cận dùng cùng bộ trọng số. Kernel có chiều sâu bằng số kênh đầu vào; không phải mỗi kênh một kernel độc lập rồi ghép tùy ý.

Suy ra kích thước không gian: gọi `n` là chiều cao (hoặc rộng), `k` là kernel, `p` là số ô zero-padding mỗi bên và `s` là stride. Sau padding có `n+2p` ô. Kernel dài `k` nên còn `n+2p−k` khoảng dịch; mỗi lần nhảy `s` ô và có vị trí đầu tiên (bước 0), vì vậy:

`output = floor((n + 2p − k) / s) + 1`
Ví dụ `n=32, k=3, p=1, s=1` (`padding="same"`): `floor((32+2−3)/1)+1 = 32`. Kernel đặt được ở vị trí bắt đầu 0, 1, ..., 31 nên có 32 vị trí. Công thức áp dụng riêng cho H và W; số kênh đầu ra lại bằng `filters` (ở ví dụ này là 16), không do công thức không gian quyết định.

2.4.2 `ReLU` – hàm kích hoạt

Áp dụng từng phần tử: `ReLU(x)=max(0,x)`, ví dụ `[-2,0.5,3]→[0,0.5,3]`. ReLU tạo phi tuyến; nếu chỉ dùng phép tuyến tính, nhiều lớp vẫn tương đương một lớp tuyến tính. Với `x>0`, gradient bằng 1 nên truyền tín hiệu tốt. ReLU không có kernel và thường giữ nguyên kích thước.

Đạo hàm: `d ReLU/dx=0` khi `x<0`, `=1` khi `x>0` (tại 0 thường quy ước 0). Vì vậy neuron âm không truyền gradient trong bước đó (dying ReLU nếu luôn âm), còn neuron dương truyền gradient. ReLU được đặt sau Conv trước Pooling; nó không giảm H,W và không có tham số.

2.4.3 `MaxPooling2D` – tổng hợp và thu nhỏ

MaxPooling cũng dùng một cửa sổ trượt, nhưng **không nhân với kernel và không học trọng số**. Tại mỗi vị trí, nó chỉ chọn giá trị lớn nhất trong cửa sổ. Ví dụ:

$$\text{Cửa sổ } 2\times2: \begin{bmatrix} 1 & 5 \\ 3 & 4 \end{bmatrix}$$

$$\text{MaxPool} = \max(1, 5, 3, 4) = 5$$

Công thức kích thước đầu ra

Gọi `H,W` là chiều cao/rộng đầu vào, `K` là `pool_size`, `S` là `strides`, `P` là padding mỗi bên:

$$H_{out} = \text{floor}((H + 2P - K) / S) + 1$$
$$W_{out} = \text{floor}((W + 2P - K) / S) + 1$$

Đây là cùng công thức không gian với convolution; chỉ phép tính trong cửa sổ khác nhau. Với `64×64×32`, `pool_size=2`, `strides=2`, `padding=0`:

$$H_{out} = \text{floor}((64 + 0 - 2)/2) + 1 = 32$$
$$W_{out} = \text{floor}((64 + 0 - 2)/2) + 1 = 32$$

Ví dụ trượt từng bước

Xét một feature map 4×4, `pool_size=2`, `stride=2`:

$$X = \begin{bmatrix} 1 & 5 & 2 & 0 \\ 3 & 4 & 1 & 2 \\ 0 & 2 & 8 & 7 \\ 1 & 3 & 6 & 9 \end{bmatrix}$$

Vị trí 1 (góc trên trái): `[[1,5],[3,4]]` → `max=5`. Dịch sang phải 2 ô, vị trí 2: `[[2,0],[1,2]]` → `max=2`. Xuống dưới 2 ô, vị trí 3: `[[0,2],[1,3]]` → `max=3`. Vị trí 4: `[[8,7],[6,9]]` → `max=9`.

$$\text{Đầu ra} = \begin{bmatrix} 5 & 2 \\ 3 & 9 \end{bmatrix}$$

`4×4` thành `2×2` vì cửa sổ rộng 2 và nhảy 2; mỗi kênh được xử lý độc lập nên `4×4×32 → 2×2×32`, số kênh không đổi. Trong backward, gradient chỉ truyền về ô lớn nhất đã thắng trong mỗi cửa sổ. AveragePooling thay vì chọn cực đại sẽ tính trung bình và truyền gradient tới mọi ô. Pooling giảm bộ nhớ/tính toán và bớt nhạy với dịch nhỏ, nhưng có thể làm mất chi tiết; một số kiến trúc dùng convolution stride 2 để thay thế.

2.4.4 `BatchNormalization` – chuẩn hóa theo batch

Theo mỗi kênh, tính `μ,σ`, chuẩn hóa `x̂=(x−μ)/√(σ²+ε)`, rồi học `γ,β`: `y=γx̂+β`. Train dùng thống kê mini-batch; inference dùng thống kê tích lũy. Kích thước thường không đổi; activation ổn định hơn và tối ưu dễ hơn.

Ví dụ: nếu activation của một kênh trong batch có `μ=10, σ=2`, giá trị `x=14` thành `x̂≈2` (bỏ qua epsilon), sau đó `y=γ×2+β`. `γ,β` học được để mạng vẫn khôi phục thang đo cần

thiết. Batch quá nhỏ làm μ,σ nhiễu; BN thường đặt sau Conv và trước/ sau ReLU tùy kiến trúc.

2.4.5 `Dropout` – chống overfitting

Khi train, tắt ngẫu nhiên neuron theo `rate` (ví dụ 0.3) và scale phần còn lại; khi evaluate, dùng toàn bộ neuron. Mạng không thể phụ thuộc một neuron duy nhất nên tổng quát tốt hơn. Dropout không đổi kích thước.

Với `rate=0.5`, mặt nạ Bernoulli có thể là `[1,0,1,0]`; inverted dropout chia các neuron còn lại cho `1-rate` khi train để kỳ vọng không đổi. Dropout không nên dùng khi cần kết quả xác định lúc inference; framework tự tắt nó trong evaluation mode.

2.4.6 `Flatten` và `GlobalAveragePooling2D`

Cả hai lớp đều nhận feature map dạng `H×W×C` và biến nó thành vector để lớp Dense/output có thể xử lý. Chúng **không có kernel và không có trọng số học được**; khác nhau ở cách giữ/thay đổi thông tin không gian.

`Flatten`: trải toàn bộ ô thành một vector

Giả sử chỉ có một kênh để dễ nhìn:

```
Feature map = ⎡ 1  2 ⎤
              ⎣ 3  4 ⎦

Flatten → [1, 2, 3, 4]
```

Với tensor `8×8×64`, Flatten tạo vector dài `8×8×64=4.096`. Mỗi ô được giữ nguyên, bao gồm thông tin vị trí theo thứ tự trải. Vì vậy Flatten giữ nhiều chi tiết hơn nhưng vector rất dài. Nếu nối Dense 128 neuron:

```
4.096 × 128 trọng số + 128 bias = 524.416 tham số
```

`GlobalAveragePooling2D` (GAP): trung bình từng feature map

GAP không trải 64×64 ô. Nó lấy trung bình **tất cả H×W ô của từng kênh**. Ví dụ một feature map:

```
M = ⎡ 1  2 ⎤        GAP(M) = (1+2+3+4)/4 = 2,5
    ⎣ 3  4 ⎦
```

Nếu có 3 kênh, GAP trả về 3 số `[mean(channel_1), mean(channel_2), mean(channel_3)]`. Vì vậy `8×8×64 → 64`. Dense 128 phía sau chỉ cần:

```
64 × 128 trọng số + 128 bias = 8.320 tham số
```

Giống nhau và khác nhau

Đặc điểm	Flatten	Global Average Pooling
Đầu vào	`H×W×C`	`H×W×C`

Đặc điểm	Flatten	Global Average Pooling
Đầu ra	Vector `H·W·C`	Vector `C`
Có học trọng số?	Không	Không
Giữ vị trí từng ô?	Có (theo thứ tự trải)	Không, chỉ giữ trung bình mỗi kênh
Ví dụ `8×8×64`	4.096 số	64 số
Dense 128 sau đó	524.416 tham số	8.320 tham số

Vì sao GAP thường ít overfitting hơn?

Overfitting xảy ra khi mô hình có quá nhiều tham số và học thuộc chi tiết riêng của ảnh train. Flatten tạo 4.096 đầu vào cho Dense; Dense có thể ghi nhớ các mẫu vị trí rất cụ thể (ví dụ một đốm sáng ở đúng tọa độ), nên nhiều tham số cần học và dễ overfit. GAP nén mỗi feature map thành một số, loại bỏ vị trí chính xác và làm Dense nhỏ hơn khoảng `524.416/8.320≈63` lần. Ít tham số hơn nghĩa là ít "dung lượng ghi nhớ" hơn, thường tổng quát tốt hơn khi dữ liệu ít.

Đổi lại, GAP có thể làm mất thông tin "đặc trưng nằm chính xác ở đâu". Nếu vị trí là quan trọng (ví dụ nhận diện bố cục), Flatten có thể hữu ích. Vì vậy đây là đánh đổi: Flatten giữ thông tin vị trí nhưng nặng; GAP nhẹ và ít overfit nhưng bất biến vị trí hơn.

2.4.7 `Dense` (fully-connected) – bộ ra quyết định

Mỗi neuron nối mọi phần tử vector: `z=Wx+b`. Sau Conv đã trích xuất đặc trưng, Dense kết hợp chúng để phân loại. Hai lớp có thể dùng `Dense(1, activation='sigmoid')` hoặc `Dense(2)` cho logits.

Nếu vector có 64 phần tử và Dense có 2 neuron, ma trận `W` có dạng `2×64`, thêm 2 bias → 130 tham số. Mỗi neuron kết hợp mọi đặc trưng; activation ở Dense ẩn thường là ReLU, còn lớp cuối dùng sigmoid/softmax tùy số lớp.

2.4.8 `Sigmoid` và binary cross-entropy

Sigmoid `p=1/(1+e^(-z))` biến logit thành xác suất 0–1. Với `y∈{0,1}`, loss `L=-[y log(p)+(1-y)log(1-p)]`: dự đoán đúng tự tin cho loss nhỏ, sai tự tin cho loss lớn. Thường chọn chó nếu `p≥0.5`.

Ví dụ: `z=0 → p=0.5`; `z=2 → p≈0.881`; `z=-2 → p≈0.119`. Nếu nhãn chó `y=1` và `p=0.9`, loss `-log(0.9)≈0.105`; nếu `p=0.1`, loss `-log(0.1)≈2.303`. Loss phạt rất mạnh dự đoán sai nhưng quá tự tin. Với nhiều hơn hai lớp, dùng softmax + categorical cross-entropy thay vì sigmoid đơn.

2.4.9 Một khối CNN hoàn chỉnh

```
Conv2D → ReLU → MaxPooling2D → (lặp lại)
       → GlobalAveragePooling hoặc Flatten
```

`→ Dropout (tùy chọn) → Dense → Sigmoid`

Conv tìm mẫu; ReLU tạo phi tuyến; Pooling thu nhỏ; BN ổn định; Dropout chống overfit; GAP/Flatten đổi dạng; Dense kết luận; Sigmoid xuất xác suất.

2.5 Huấn luyện

Forward pass tạo dự đoán; loss đo sai số; backpropagation tính đạo hàm; optimizer (SGD/Adam) cập nhật trọng số `w ← w − η∇w`. Một epoch là một lượt qua toàn bộ train set. Validation dùng để theo dõi khả năng tổng quát; test chỉ dùng cuối cùng.

2.6 Các mô hình cải tiến từ CNN

Các mô hình dưới đây vẫn dùng những viên gạch Conv–Activation–Pooling, nhưng cải tiến cách xếp lớp để mạng sâu hơn, chính xác hơn hoặc nhẹ hơn. Không có mô hình nào "thay thế hoàn toàn" CNN; chúng là các kiến trúc CNN được thiết kế tốt hơn cho những vấn đề khác nhau.

2.6.1 LeNet-5 – nền móng của CNN

Đầu ra: LeNet-5 gốc kết thúc bằng `Dense(10)` vì MNIST có 10 chữ số. Với chó/mèo, thay thành `Dense(1, sigmoid)` (output `(batch,1)`, binary cross-entropy) hoặc `Dense(2)` (output `(batch,2)`, CrossEntropyLoss).

LeNet-5 được thiết kế để nhận dạng chữ số viết tay. Ảnh đầu vào thường được chuẩn hóa thành `32×32×1` (cao × rộng × 1 kênh xám). Kiến trúc đầy đủ:

```
Input 32×32×1
  ↓
C1: Conv 6 kernel 5×5       → 28×28×6
  ↓
S2: Average Pool 2×2, stride 2 → 14×14×6
  ↓
C3: Conv 16 kernel 5×5      → 10×10×16
  ↓
S4: Average Pool 2×2, stride 2 → 5×5×16
  ↓
Flatten                      → 400
  ↓
Dense                        → 120
  ↓
Dense                        → 84
  ↓
Output Dense                 → 10 chữ số (0–9)
```

Lớp C1 – Convolution: 6 kernel `5×5×1` quét ảnh. Không padding nên kích thước là `32−5+1=28`; 6 kernel tạo 6 feature map `28×28`. Mỗi kernel học một mẫu nét khác nhau.

Lớp S2 – Average Pooling: đây là pooling lấy **trung bình**, không lấy cực đại như MaxPool. Với mỗi cửa sổ `2×2`, tính `(a+b+c+d)/4`; ví dụ `[[1,5],[3,4]]` cho `3,25`. `stride=2`

làm cửa sổ không chồng lấn nên `28×28→14×14`; xử lý riêng từng kênh nên vẫn có 6 kênh. Average Pool làm mượt và giảm kích thước.

Lớp C3 – Convolution: 16 kernel `5×5×6` (chiều sâu 6 vì đầu vào có 6 feature map) quét `14×14×6`; không padding cho `14−5+1=10`, tạo `10×10×16`. Mỗi kernel kết hợp thông tin từ cả 6 kênh, nên bắt đầu ghép các nét đơn giản.

Lớp S4 – Average Pooling: lại lấy trung bình từng vùng `2×2`, stride 2: `10×10×16→5×5×16`. Các đặc trưng được tóm tắt, bớt nhạy với dịch nhỏ của nét chữ.

Flatten: trải `5×5×16` thành vector `5×5×16=400` số. Đây là bước chuyển từ feature map sang dạng phù hợp với Dense.

Dense 120 và Dense 84: mỗi neuron nối với toàn bộ vector trước đó, kết hợp các nét để tạo biểu diễn phức tạp hơn. Đây là phần fully-connected của LeNet.

Output Dense 10: tạo 10 điểm số, một điểm cho mỗi chữ số `0,1,...,9`; chọn điểm cao nhất (hoặc dùng softmax để đổi thành xác suất).

LeNet-5 nhỏ và dễ học, chứng minh Conv + Pooling hiệu quả hơn Dense thuần túy. Tuy nhiên nó chỉ có vài lớp và ít filters, nên không đủ mạnh cho ảnh chó/mèo RGB lớn.

2.6.2 AlexNet – đưa CNN vào ảnh lớn

Đầu ra: AlexNet gốc có `Dense(1000)` cho 1000 lớp ImageNet; logits có dạng `(batch,1000)`. Với chó/mèo, giữ toàn bộ Conv/Pool, thay Dense cuối thành `Dense(1,sigmoid)` hoặc `Dense(2)`; cũng nên thay hai Dense 4096 nếu cần giảm tham số.

Đi từng phần: (1) `Input 227×227×3`: ảnh RGB. (2) `Conv11×11,96,stride4`: quét vùng lớn, giảm còn khoảng `55×55×96`; ReLU giữ phản hồi dương. (3) `MaxPool3×3,stride2`: còn khoảng `27×27×96`. (4) Các Conv `5×5` rồi `3×3` lần lượt tăng/đổi số kênh `256→384→384→256`; mỗi lớp học đặc trưng mới từ feature map trước. (5) MaxPool cuối giảm còn khoảng `6×6×256`. (6) Flatten tạo `6×6×256=9.216` số; Dense 4096 kết hợp chúng, Dropout ngẫu nhiên tắt neuron lúc train; Dense 4096 tiếp tục kết hợp. (7) Dense 1000 tạo một điểm số cho mỗi lớp ImageNet.

```
Input 227×227×3
→ Conv 11×11, 96, stride 4 → ReLU → MaxPool 3×3
→ Conv 5×5, 256 → ReLU → MaxPool
→ Conv 3×3, 384 → ReLU
→ Conv 3×3, 384 → ReLU
→ Conv 3×3, 256 → ReLU → MaxPool
→ Flatten → Dense 4096 → Dropout → Dense 4096 → Dropout → Dense 1000
```

Conv đầu giảm nhanh kích thước và học cạnh/màu; các Conv sau ghép thành texture/hình dạng. MaxPool giữ phản hồi mạnh và giảm H,W. Ba Dense cuối kết hợp đặc trưng để dự đoán 1.000 lớp ImageNet. AlexNet nổi bật vì dùng ReLU (tối ưu nhanh hơn sigmoid),

Dropout và augmentation (giảm overfit), GPU (tính convolution song song). Nhược điểm là khoảng 60 triệu tham số, chủ yếu ở Dense, nên nặng hơn các kiến trúc sau.

2.6.3 VGG-16/VGG-19 – sâu bằng các Conv 3×3 lặp lại

Đầu ra: VGG ImageNet đi từ `7×7×512 → Flatten(25088) → Dense(4096) → Dense(4096) → Dense(1000)`. Bỏ `Dense(1000)` và gắn `Dense(1,sigmoid)` / `Dense(2)`. Có thể bỏ toàn bộ head Dense lớn, dùng `GlobalAveragePooling2D → Dense(1)` để giảm mạnh tham số.

Đọc từng block: Block 1 nhận `224×224×3`, hai Conv tạo `224×224×64`, Pool còn `112×112×64`. Block 2 dùng 128 filters, Pool còn `56×56×128`. Block 3 dùng ba Conv 256, Pool còn `28×28×256`. Block 4 và 5 dùng ba Conv 512, lần lượt Pool còn `14×14×512` rồi `7×7×512`. Cuối cùng Flatten `7×7×512=25.088`, Dense 4096 → Dense 4096 → Output 1000. Mỗi Pool giảm H,W; mỗi Conv tăng số loại đặc trưng/kênh.

VGG chia mạng thành 5 block; mỗi block gồm Conv `3×3 + ReLU`, cuối block là MaxPool `2×2, stride 2`:

```
Input 224×224×3
→ [Conv64, Conv64, Pool]       112×112×64
→ [Conv128, Conv128, Pool]       56×56×128
→ [Conv256×3, Pool]              28×28×256
→ [Conv512×3, Pool]              14×14×512
→ [Conv512×3, Pool]                7×7×512
→ Flatten → Dense4096 → Dense4096 → Output1000
```

VGG-16 có 13 Conv + 3 Dense = 16 lớp có trọng số; VGG-19 có 16 Conv + 3 Dense. Hai Conv 3×3 liên tiếp có receptive field gần 5×5 nhưng thêm ReLU ở giữa, giúp học phi tuyến tốt hơn. Kiến trúc đơn giản, dễ hiểu/transfer learning, nhưng Dense 4096 làm VGG-16 khoảng 138 triệu tham số và tốn bộ nhớ.

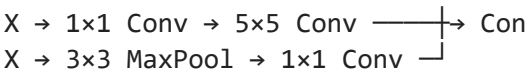
2.6.4 Inception/GoogLeNet – nhìn nhiều kích thước cùng lúc

Đầu ra: GoogLeNet thường kết thúc `Global Average Pooling → Linear(1024,1000)`, nên output `(batch,1000)`. Với 2 lớp, thay Linear bằng `Linear(1024,2)` hoặc một logit `Linear(1024,1)`; backbone Inception giữ nguyên.

Một module cụ thể: cùng tensor `28×28×192` đi vào bốn nhánh. Nhánh A Conv1×1 tạo `28×28×64`; nhánh B Conv1×1 giảm kênh rồi Conv3×3 tạo `28×28×128`; nhánh C Conv1×1 rồi Conv5×5 tạo `28×28×32`; nhánh D MaxPool3×3 giữ không gian rồi Conv1×1 tạo `28×28×32`. Concatenate theo kênh cho `28×28×256`. Module sau nhận tensor này và lặp lại; cuối mạng Global Average Pooling biến mỗi kênh thành một số rồi classifier.

Thay vì chọn một kernel duy nhất, mỗi **Inception module** tách đầu vào thành bốn nhánh rồi ghép theo chiều kênh:

```
X → 1×1 Conv ──────────────────┐
X → 1×1 Conv → 3×3 Conv ──────┤
```



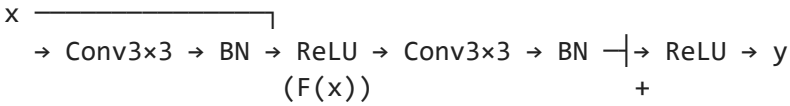
Nhánh 1×1 trộn kênh; các 1×1 trước 3×3/5×5 giảm kênh (bottleneck), nhờ vậy ít phép tính. Nhánh nhỏ bắt chi tiết, nhánh lớn bắt mẫu rộng, pooling giữ thông tin bền vững. GoogLeNet xếp nhiều module, thêm classifier phụ khi train để gradient truyền tốt hơn; khoảng 5 triệu tham số nhưng thiết kế phức tạp.

2.6.5 ResNet – skip connection để train mạng rất sâu

Đầu ra: ResNet18 thường có `GAP → Linear(512,1000)`; ResNet50 bottleneck có vector cuối 2048 rồi `Linear(2048,1000)`. Thay `model.fc` thành `Linear(512,2)` / `Linear(2048,2)` (PyTorch) hoặc Dense tương ứng (Keras). Output hai logits có dạng `(batch,2)`.

Một residual block: Input `56×56×64` đi qua Conv3×3 → BN → ReLU vẫn `56×56×64`, rồi Conv3×3 → BN vẫn dạng đó. Nhánh shortcut giữ nguyên `x`; phép cộng từng ô `y=F(x)+x`, sau đó ReLU. Khi cần giảm còn `28×28×128`, Conv đầu stride2 và shortcut Conv1×1,stride2 cùng đổi kích thước để cộng được. Nhiều block nối tiếp tạo ResNet-18/34/50; cuối mạng Pool/Global Average Pooling → Dense.

Một residual block nhận `x`, tính nhánh biến đổi `F(x)`, rồi cộng shortcut:



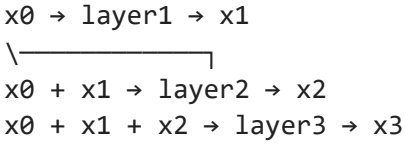
Nếu kích thước/kênh thay đổi, shortcut dùng Conv `1×1` để chiếu `x` về cùng dạng. Nhờ phép cộng, gradient có đường đi ngắn trực tiếp; block chỉ cần học phần thay đổi `F(x)` thay vì học toàn bộ ánh xạ. ResNet-18/34 dùng basic block; ResNet-50/101 dùng bottleneck `1×1→3×3→1×1`. Đây là backbone mạnh cho transfer learning.

2.6.6 DenseNet – mỗi lớp nhận mọi đặc trưng trước đó

Đầu ra: DenseNet121 thường có `GAP → Linear(1024,1000)`. Thay classifier bằng `Linear(1024,2)` hoặc `Linear(1024,1)`; các dense block/transition không cần đổi. Output phải khớp loss và nhãn 0/1.

Ví dụ growth rate `k=32`: đầu vào block có 64 kênh. Layer 1 tạo 32 kênh mới, concatenate → 96; layer 2 nhận 96 và tạo 32 → 128; layer 3 nhận 128 và tạo 32 → 160. Transition `1×1 Conv` giảm kênh rồi AveragePool stride2 giảm H,W. Các lớp không thay thế đặc trưng cũ mà giữ và ghép chúng, nên layer sau nhìn được toàn bộ lịch sử.

Trong một dense block, đầu vào của lớp sau là phép **ghép kênh** tất cả đầu ra trước:



“+” ở đây là concatenate, không phải cộng số như ResNet. Mỗi layer thường là BN→ReLU→Conv `1×1` (bottleneck)→Conv `3×3`; `growth rate k` là số kênh mới mỗi layer. Transition layer dùng `1×1 Conv + AveragePool` để giảm kênh/kích thước giữa các block. Tái sử dụng đặc trưng và nhiều đường gradient giúp ít tham số, nhưng tensor ghép lớn tốn RAM.

2.6.7 MobileNet – CNN nhẹ cho điện thoại

Đầu ra: MobileNetV2 thường có `GAP → Dropout → Dense(1280,1000)`. Khi dùng `include_top=False`, backbone trả feature map; thêm GAP rồi `Dense(1,sigmoid)` cho chó/mèo. Đây là lựa chọn nhẹ, phù hợp ảnh và thiết bị hạn chế.

Một block: Input `32×32×32` → Depthwise Conv3×3 (mỗi 32 kênh một kernel, vẫn 32 kênh) → BN/ReLU6 → Pointwise Conv1×1 (trộn 32 kênh và tạo 64) → BN/ReLU6, đầu ra `32×32×64`. Nếu stride2, Depthwise đồng thời giảm còn `16×16×32`. Các block lặp lại; cuối Global Average Pooling → Dense nhỏ.

MobileNet thay Conv thường bằng **depthwise separable convolution**:

```
Input H×W×C_in → Depthwise k×k (mỗi kênh một kernel)
                → Pointwise 1×1 (trộn kênh)
                → H×W×C_out
```

Depthwise không trộn kênh nên rẻ; pointwise `1×1` tạo số kênh mới. Ví dụ `3×3, C_in=32, C_out=64`: Conv thường 18.432 trọng số; depthwise + pointwise chỉ `3×3×32 + 32×64 = 2.336`. MobileNetV1 dùng ReLU6 và các block này; các bản V2/V3 thêm inverted residual, linear bottleneck, squeeze-and-excitation. Nhanh/nhẹ cho mobile nhưng accuracy thường thấp hơn backbone lớn.

2.6.8 EfficientNet – mở rộng cân bằng

Đầu ra: EfficientNet-B0 thường đưa vector 1280 vào classifier 1000 lớp. Khi bỏ head ImageNet, dùng `GAP → Dropout → Dense(1,sigmoid)` hoặc `Dense(2)`. Các bản B1–B7 thay đổi kích thước/chi phí backbone nhưng cách đổi lớp cuối giống nhau.

Một MBConv: Pointwise 1×1 mở rộng số kênh → Depthwise 3×3/5×5 học không gian → Squeeze-and-Excitation tính trọng số quan trọng cho từng kênh → Pointwise 1×1 chiếu về số kênh đầu ra → cộng shortcut nếu cùng kích thước. Các MBConv xếp thành nhiều stage, mỗi stage có thể giảm H,W và tăng kênh. B0 dùng ít stage/filters; B1–B7 tăng đồng thời số block, số kênh và độ phân giải ảnh theo compound scaling.

EfficientNet bắt đầu bằng backbone nhỏ B0 (các MBConv depthwise + squeeze-and-excitation + skip connection), sau đó tạo B1…B7 bằng **compound scaling**. Thay vì chỉ tăng một yếu tố, khi tăng hệ số ϕ, mạng đồng thời tăng có kiểm soát:

```
depth  (số block)   ∝ α^ϕ
width  (số kênh)    ∝ β^ϕ
resolution (H,W)    ∝ γ^ϕ
```

Các hệ số α, β, γ được chọn qua thực nghiệm sao cho chi phí tăng hợp lý. B0 nhỏ/nhanh; B7 lớn/chính xác hơn nhưng chậm hơn. Ưu điểm là accuracy trên mỗi tham số tốt; nhược điểm là nhiều block và quy tắc scaling khó hiểu hơn VGG/ResNet.

2.6.9 Bảng so sánh nhanh

Mô hình	Ý tưởng chính	Điểm mạnh	Hạn chế
LeNet-5	CNN nhỏ đầu tiên	Dễ học, nhẹ	Không hợp ảnh lớn
AlexNet	ReLU, GPU, dropout	Bước ngoặt ImageNet	Nhiều tham số
VGG	Nhiều Conv 3×3	Đơn giản, dễ transfer	Rất nặng
Inception	Nhánh song song	Nhiều kích thước, ít tham số	Phức tạp
ResNet	Skip connection	Train mạng rất sâu	Cần thiết kế shortcut
DenseNet	Concatenate đặc trưng	Tái sử dụng, gradient tốt	Tốn memory
MobileNet	Depthwise separable	Rất nhẹ, nhanh	Có thể giảm accuracy
EfficientNet	Compound scaling	Hiệu quả tham số	Khó hiểu/cài đặt hơn

Với bài chó/mèo, có thể bắt đầu bằng CNN nhỏ tự xây để hiểu cơ chế; khi cần accuracy cao và ít dữ liệu, dùng ResNet/MobileNet/EfficientNet pretrained rồi fine-tune.

2.7 Transfer learning

Dùng backbone đã học trên ImageNet, đóng băng phần lớn lớp và huấn luyện classifier chó/mèo; sau đó có thể fine-tune một số lớp cuối với learning rate nhỏ. Cách này cần ít dữ liệu và thời gian hơn CNN học hoàn toàn từ đầu.

2.8 Đánh giá và lỗi thường gặp

Accuracy phù hợp khi hai lớp cân bằng; precision, recall, F1 và confusion matrix hữu ích khi dữ liệu lệch lớp. Không được trộn ảnh test vào train; mọi phép chuẩn hóa phải nhất quán; augmentation chỉ áp dụng train; luôn lưu seed và kiểm tra nhãn bằng cách hiển thị ảnh.

2.4.10 Theo dõi kích thước và số tham số qua một CNN

Giả sử ảnh đầu vào `128×128×3` (cao × rộng × kênh màu):

```
Input                        128×128×3
Conv2D(32, 3×3, same)        128×128×32
ReLU                         128×128×32
MaxPool(2×2, stride 2)         64×64×32
Conv2D(64, 3×3, same)          64×64×64
MaxPool(2×2, stride 2)          32×32×64
GlobalAveragePooling2D                64
Dense(1, sigmoid)                      1
```

Conv đầu tiên: mỗi kernel có `3×3×3=27` trọng số và 1 bias; 32 kernel có `32×(27+1)=896` tham số. Conv thứ hai: mỗi kernel có `3×3×32=288` trọng số; 64 kernel có `64×(288+1)=18.496` tham số. Pooling giảm H,W; số kênh tăng vì mạng học nhiều loại đặc trưng hơn.

2.4.11 Tham số học được và siêu tham số

Học được: trọng số/bias của Conv, `γ/β` của BatchNorm, trọng số/bias của Dense; optimizer cập nhật chúng bằng gradient. **Do người chọn:** số lớp, số filters, kernel size, stride, padding, pooling size, dropout rate, learning rate, batch size. Người không gán nội dung "kernel này là cạnh"; người thiết kế không gian để kernel tự học.

2.4.12 Train mode và evaluation mode

Conv, ReLU, Pooling gần như giống nhau ở train và dự đoán. BatchNorm train dùng thống kê batch, dự đoán dùng thống kê tích lũy. Dropout chỉ tắt neuron khi train; lúc evaluate/predict phải tắt. Vì vậy Keras `fit/evaluate` và PyTorch `model.train()/model.eval()` là các chế độ khác nhau.

2.4.13 Lỗi thường gặp

- `filters=32` là 32 kernel/feature map, không phải 32 neuron Dense.
- Pooling chỉ chọn cực đại, không có trọng số để học.
- Conv tạo phản hồi; ReLU chỉ biến đổi phản hồi, không tự phát hiện cạnh.
- Kernel Conv RGB có chiều sâu 3; ở lớp sau chiều sâu bằng số feature map trước.
- Flatten quá sớm tạo vector dài và Dense nhiều tham số.
- Phải dùng evaluation mode khi đánh giá để tắt Dropout.

2.6.10 Đầu ra ImageNet và cách đổi sang bài toán chó/mèo

Các backbone được công bố thường đi kèm **classification head** đã học trên ImageNet (1000 lớp):

```
Ảnh → backbone Conv (ResNet/MobileNet/EfficientNet)
    → Global Average Pooling
    → Dense/Linear 1000
    → logits có dạng (batch_size, 1000)
```

Con số `1000` là số lớp của ImageNet, **không phải** số lớp cố định của bản thân ResNet hay MobileNet. Khi áp dụng cho thư mục `cats/dogs`, ta bỏ head 1000 lớp và thay head mới:

Cách 1 – một logit + sigmoid (nhị phân):

```
backbone → GAP (vector 1280 chẳng hạn) → Dense(1) → sigmoid
                                         output: (batch_size, 1)
```

Giá trị sigmoid là xác suất lớp có nhãn 1; dùng `binary_crossentropy`. Trong Keras:

```
base = tf.keras.applications.MobileNetV2(include_top=False,
weights="imagenet", input_shape=(128,128,3))
base.trainable = False
x = tf.keras.layers.GlobalAveragePooling2D()(base.output)
output = tf.keras.layers.Dense(1, activation="sigmoid")(x)
model = tf.keras.Model(base.input, output)
```

Cách 2 – hai logits + softmax/CrossEntropyLoss:

```
backbone → GAP (vector D) → Linear(D, 2)
                            output: (batch_size, 2)
```

Hai số là điểm cho `cat` và `dog`; chọn chỉ số lớn hơn. Trong PyTorch:

```
weights = torchvision.models.ResNet18_Weights.DEFAULT
model = torchvision.models.resnet18(weights=weights)
in_features = model.fc.in_features
model.fc = torch.nn.Linear(in_features, 2)
```

Dùng `CrossEntropyLoss` (không thêm softmax trong model vì loss tự xử lý logits).

`include_top=False` /thay `model.fc` nghĩa là giữ lại backbone để trích xuất đặc trưng, chỉ thay phần phân loại. Ban đầu thường đóng băng backbone và train head mới; khi head ổn định có thể mở một vài block cuối, fine-tune với learning rate nhỏ. Nếu quên thay head, model vẫn trả 1000 logits và không khớp nhãn chó/mèo. Số chiều vector trước head phụ thuộc backbone (ResNet18 thường 512, MobileNetV2 1280, EfficientNet-B0 1280), nhưng số lớp cuối phải đổi thành 1 hoặc 2 theo cách chọn ở trên.

In [1]:
```
from pathlib import Path

DATA_DIR = Path.cwd()
if not (DATA_DIR / "train").exists() and (DATA_DIR / "Dog-Cat" / "train").exists():
    DATA_DIR = DATA_DIR / "Dog-Cat"
TRAIN_DIR = DATA_DIR / "train"
TEST_DIR = DATA_DIR / "test"

assert TRAIN_DIR.exists(), f"Không tìm thấy: {TRAIN_DIR}"
assert TEST_DIR.exists(), f"Không tìm thấy: {TEST_DIR}"

print("Train directory:", TRAIN_DIR)
print("Test directory:", TEST_DIR)
```

```
Train directory: C:\Users\ASUS\Code\Assigned_4\Dog-Cat\train
Test directory: C:\Users\ASUS\Code\Assigned_4\Dog-Cat\test
```

Giải thích cell cấu hình đường dẫn

`Path.cwd()` lấy thư mục chứa notebook khi kernel chạy. Từ đó tạo `TRAIN_DIR` và `TEST_DIR`, trỏ đến hai thư mục dữ liệu. `assert` dừng sớm với thông báo rõ ràng nếu đường dẫn sai; các cell khảo sát và huấn luyện phía sau đều dùng chung hai biến này.

Khảo sát dữ liệu trước khi xây CNN

Trước khi lập trình model, ta cần nhìn ảnh thật để kiểm tra ảnh có đọc đúng, nhãn có đúng và hai lớp có bị mất cân bằng hay không.

In [2]:
```python
def load_image_paths(folder):
    """Quét đệ quy và trả về danh sách đường dẫn ảnh."""
    extensions = {".jpg", ".jpeg", ".png", ".bmp"}
    return sorted(
        path for path in Path(folder).rglob("*")
        if path.suffix.lower() in extensions
    )

all_train_paths = load_image_paths(TRAIN_DIR)
test_paths = load_image_paths(TEST_DIR)

print(f"Số ảnh train: {len(all_train_paths)}")
print(f"Số ảnh test: {len(test_paths)}")
```

```
Số ảnh train: 20000
Số ảnh test: 5000
```

Giải thích cell load dữ liệu

Đây là bước **load danh sách dữ liệu**, chưa đưa toàn bộ ảnh vào RAM. `Path.rglob` quét các thư mục con; mỗi phần tử trong `train_paths` / `test_paths` là một đường dẫn tới file ảnh. Ảnh chỉ được mở khi cell hiển thị cần dùng, giúp tiết kiệm bộ nhớ. Việc in số lượng giúp kiểm tra đường dẫn và dữ liệu đã đọc đúng.

In [3]:
```python
import random
import matplotlib.pyplot as plt
from PIL import Image

def show_sample_images(train_dir=TRAIN_DIR, n_images=5, seed=42):
    """Hiển thị n_images ảnh train và nhãn từ tên thư mục."""
    random.seed(seed)
    image_paths = all_train_paths
    samples = random.sample(image_paths, k=min(n_images, len(image_paths)))
    columns = 5
    rows = (len(samples) + columns - 1) // columns
    plt.figure(figsize=(15, 3 * rows))
    for i, image_path in enumerate(samples):
        image = Image.open(image_path).convert("RGB")
        label = image_path.parent.name
        plt.subplot(rows, columns, i + 1)
        plt.imshow(image)
        plt.title(f"Nhãn: {label}")
        plt.axis("off")
    plt.tight_layout()
    plt.show()

show_sample_images()
```



In [4]:
```
import numpy as np
import matplotlib.pyplot as plt
train_cat = sum(path.parent.name.lower() == "cats" for path in all_train_paths)
train_dog = sum(path.parent.name.lower() == "dogs" for path in all_train_paths)
test_cat = sum(path.parent.name.lower() == "cats" for path in test_paths)
test_dog = sum(path.parent.name.lower() == "dogs" for path in test_paths)

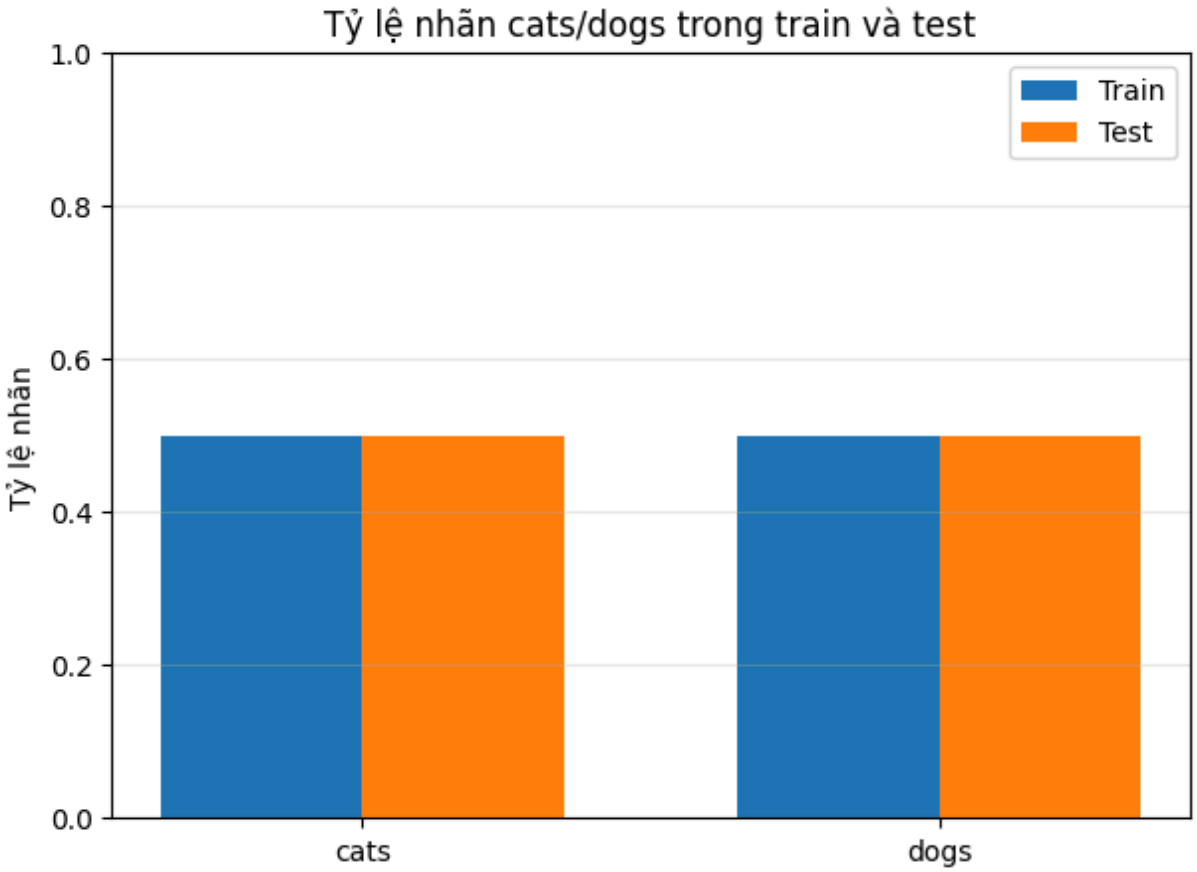
train_total = train_cat + train_dog
test_total = test_cat + test_dog

train_values = [train_cat / train_total, train_dog / train_total]
test_values = [test_cat / test_total, test_dog / test_total]
labels = ["cats", "dogs"]
x = np.arange(len(labels))
width = 0.35

print(f"Train: cats={train_cat} ({train_values[0]:.2%}), dogs={train_dog} ({train_v
print(f"Test:  cats={test_cat} ({test_values[0]:.2%}), dogs={test_dog} ({test_value

plt.figure(figsize=(7, 5))
plt.bar(x - width / 2, train_values, width, label="Train")
plt.bar(x + width / 2, test_values, width, label="Test")
plt.xticks(x, labels)
plt.ylabel("Tỷ lệ nhãn")
plt.ylim(0, 1)
plt.title("Tỷ lệ nhãn cats/dogs trong train và test")
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.show()
```

```
Train: cats=10000 (50.00%), dogs=10000 (50.00%)
Test:  cats=2500 (50.00%), dogs=2500 (50.00%)
```



Tỉ lệ nhãn cân bằng trong cả tập train và test

Lưu kết quả để so sánh ba model

Hãy chạy các cell train trước, sau đó chạy từng cell lưu kết quả. Các cell lưu cũng tự tạo `results` nếu bạn chạy riêng.

In [5]:
```
results = []
histories = {}
```

In [6]:
```
rng = np.random.default_rng(42)
indices = rng.permutation(len(all_train_paths))
split = int(0.8 * len(all_train_paths))
train_paths = [all_train_paths[i] for i in indices[:split]]
val_paths = [all_train_paths[i] for i in indices[split:]]
```

3. CNN từ scratch bằng NumPy

Kkiến trúc hai convolution block: `Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Dense → ReLU → Dense → Softmax`. Tự viết forward/backward và Adam, không dùng autograd.

In [8]:
```python
import numpy as np
from PIL import Image
```

Chuẩn bị dữ liệu NumPy

Ảnh được đọc từ `train_paths` / `test_paths`, resize về 32×32, chuẩn hóa [0,1] và gán `cats=0`, `dogs=1`. 80% train dùng học, 20% validation.

In [7]:
```python
def load_numpy(paths, image_size=(32, 32)):
    """Đọc danh sách đường dẫn thành mảng ảnh và nhãn NumPy."""
    images = []
    labels = []
    for path in paths:
        image = Image.open(path).convert("RGB").resize(image_size)
        images.append(np.asarray(image, dtype=np.float32) / 255.0)
        labels.append(0 if path.parent.name.lower() == "cats" else 1)
    return np.stack(images), np.asarray(labels, dtype=np.int64)
```

Giải thích `load_numpy`

Hàm nhận danh sách đường dẫn đã chia sẵn, mở từng ảnh bằng PIL, đổi RGB, resize về 32×32, chuẩn hóa pixel về [0,1] và lấy nhãn từ thư mục `cats` / `dogs`. `np.stack` ghép ảnh thành tensor `(N,H,W,C)`; hàm không tự chia lại dữ liệu.

In [9]:
```python
x_train_numpy, y_train_numpy = load_numpy(train_paths)
x_val_numpy, y_val_numpy = load_numpy(val_paths)
x_test_numpy, y_test_numpy = load_numpy(test_paths)
```

In [10]:
```python
class Conv2D:
    def __init__(
        self,
        in_channels,
        out_channels,
        kernel_size=3,
        padding=1,
        random_state=42,
    ):
        self.kernel_size = kernel_size
        self.padding = padding
        rng = np.random.default_rng(random_state)
        fan_in = kernel_size * kernel_size * in_channels
        self.weight = rng.normal(
            0.0,
            np.sqrt(2.0 / fan_in),
            size=(kernel_size, kernel_size, in_channels, out_channels),
        ).astype(np.float32)
        self.bias = np.zeros(out_channels, dtype=np.float32)

    def _extract_patches(self, x):
        p = self.padding
        x_padded = np.pad(
```

```
            x,
            ((0, 0), (p, p), (p, p), (0, 0)),
            mode="constant",
        )
        windows = np.lib.stride_tricks.sliding_window_view(
            x_padded,
            (self.kernel_size, self.kernel_size),
            axis=(1, 2),
        )
        return x_padded, windows.transpose(0, 1, 2, 4, 5, 3)

    def forward(self, x):
        self.x_padded, self.patches = self._extract_patches(x)
        return (
            np.einsum(
                "nhwklc,klco->nhwo",
                self.patches,
                self.weight,
                optimize=True,
            )
            + self.bias
        )

    def backward(self, grad_output):
        self.grad_weight = np.einsum(
            "nhwklc,nhwo->klco",
            self.patches,
            grad_output,
            optimize=True,
        )
        self.grad_bias = grad_output.sum(axis=(0, 1, 2))
        grad_patches = np.einsum(
            "nhwo,klco->nhwklc",
            grad_output,
            self.weight,
            optimize=True,
        )

        grad_padded = np.zeros_like(self.x_padded)
        out_h, out_w = grad_output.shape[1:3]
        for row in range(self.kernel_size):
            for col in range(self.kernel_size):
                grad_padded[
                    :, row:row + out_h, col:col + out_w, :
                ] += grad_patches[:, :, :, row, col, :]

        p = self.padding
        return grad_padded if p == 0 else grad_padded[:, p:-p, p:-p, :]
```

Giải thích lớp `Conv2D` – forward, backward và cập nhật trọng số

`Conv2D` là lớp học các **kernel** để biến ảnh đầu vào thành feature map. Kernel/weight là những con số mà model phải học; ban đầu chúng được khởi tạo nhỏ theo He initialization,

chưa biết cạnh hay hình dạng nào.

1. Forward là gì?

Forward (lan truyền thuận) là quá trình đi từ ảnh đến dự đoán. Với mỗi batch, `forward(x)`:

1. Thêm padding 0 quanh ảnh nếu `padding=1`.
2. Cắt các vùng ảnh có kích thước bằng kernel.
3. Nhân từng pixel trong vùng với trọng số kernel, cộng tất cả và cộng bias.
4. Lặp ở mọi vị trí để tạo feature map.

Ví dụ một vùng ảnh và kernel 2×2:

```
Vùng ảnh P = ⎡ 1  2 ⎤   Kernel W = ⎡ 0.5  0 ⎤   bias = 0
             ⎣ 3  4 ⎦              ⎣ 0    1 ⎦
```

Giá trị output tại vị trí này là `1×0.5 + 2×0 + 3×0 + 4×1 = 4.5`. Trong code, `self.patches` lưu các vùng ảnh để backward dùng lại; `np.einsum` thực hiện các phép nhân–cộng cho nhiều vị trí cùng lúc.

2. Trọng số và loss

Trọng số (`self.weight`) và bias quyết định output. Sau forward, các lớp sau tạo logits; loss so sánh logits với nhãn thật. Loss lớn nghĩa là dự đoán sai/nhiều; mục tiêu train là tìm weight làm loss nhỏ.

3. Gradient là gì?

Gradient là đạo hàm của loss theo một trọng số: nó cho biết nếu tăng trọng số một chút thì loss tăng hay giảm bao nhiêu. Ví dụ `gradient=+2` nghĩa là tăng weight làm loss tăng, nên cần giảm weight; `gradient=-0.5` nghĩa là tăng weight làm loss giảm, nên cần tăng weight.

4. Backward là gì?

Backward/backpropagation đi ngược từ loss về các lớp trước để tính gradient bằng quy tắc chain rule. Trong `Conv2D.backward(grad_output)`:

- `grad_output` là gradient từ lớp phía sau truyền vào.
- `grad_weight` cho biết mỗi kernel weight cần thay đổi thế nào; code tính bằng tổng `patch × grad_output` trên mọi ảnh/vị trí.
- `grad_bias` là tổng gradient theo các vị trí.
- Gradient theo input được tính bằng cách phân phối ảnh hưởng ngược qua kernel để lớp trước tiếp tục nhận gradient. Backward **chưa cập nhật** weight; nó chỉ tính và lưu gradient.

5. Optimizer cập nhật ra sao?

Sau backward, optimizer (ở notebook là `NumpyAdam`) đọc từng cặp `(weight, gradient)` và cập nhật:

`weight mới = weight cũ − learning_rate × hướng cập nhật`
Adam không dùng gradient thô ngay lập tức; nó lưu trung bình động `m` của gradient và `v` của gradient bình phương, rồi chia cho `sqrt(v)+epsilon` để bước cập nhật ổn định hơn. Sau cập nhật, kernel thay đổi một chút. Lặp lại forward → loss → backward → optimizer.step trên nhiều batch/epoch khiến kernel dần phù hợp với dữ liệu.

Chuỗi đầy đủ:

```
weight ban đầu
→ forward tạo feature map/logits
→ tính loss
→ backward tính gradient
→ Adam cập nhật weight/bias
→ lặp lại
```
CNN không được lập trình trước kernel nào là cạnh; dữ liệu và gradient quyết định kernel học mẫu gì.

In [11]:
```python
class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return np.maximum(x, 0)

    def backward(self, grad_output):
        return grad_output * self.mask
```

Giải thích `ReLU`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [12]:
```python
class MaxPool2D:
    def __init__(self, pool_size=2):
        self.pool_size = pool_size

    def forward(self, x):
        n, h, w, c = x.shape
        p = self.pool_size
        if h % p != 0 or w % p != 0:
            raise ValueError("Chiều cao/rộng phải chia hết cho pool_size.")
        self.input_shape = x.shape
        self.blocks = x.reshape(n, h // p, p, w // p, p, c)
        output = self.blocks.max(axis=(2, 4))
        self.max_mask = self.blocks == output[:, :, None, :, None, :]
        self.tie_count = self.max_mask.sum(axis=(2, 4), keepdims=True)
        return output
```

```
    def backward(self, grad_output):
        distributed = (
            self.max_mask
            * grad_output[:, :, None, :, None, :]
            / self.tie_count
        )
        return distributed.reshape(self.input_shape)
```

Giải thích `MaxPool2D`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [13]:
```
class Flatten:
    def forward(self, x):
        self.input_shape = x.shape
        return x.reshape(x.shape[0], -1)

    def backward(self, grad_output):
        return grad_output.reshape(self.input_shape)
```

Giải thích `Flatten`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [14]:
```
class Linear:
    def __init__(self, in_features, out_features, random_state=43):
        rng = np.random.default_rng(random_state)
        self.weight = rng.normal(
            0.0,
            np.sqrt(2.0 / in_features),
            size=(in_features, out_features),
        ).astype(np.float32)
        self.bias = np.zeros(out_features, dtype=np.float32)

    def forward(self, x):
        self.input = x
        return x @ self.weight + self.bias

    def backward(self, grad_output):
        self.grad_weight = self.input.T @ grad_output
        self.grad_bias = grad_output.sum(axis=0)
        return grad_output @ self.weight.T
```

Giải thích `Linear`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [15]:
```python
class SoftmaxCrossEntropy:
    def forward(self, logits, labels):
        shifted = logits - logits.max(axis=1, keepdims=True)
        exp_values = np.exp(shifted)
        self.probabilities = exp_values / exp_values.sum(
            axis=1,
            keepdims=True,
        )
        self.labels = labels.astype(np.int64)
        correct = self.probabilities[
            np.arange(len(labels)),
            self.labels,
        ]
        return float(-np.log(np.clip(correct, 1e-9, 1.0)).mean())

    def backward(self):
        gradient = self.probabilities.copy()
        gradient[np.arange(len(self.labels)), self.labels] -= 1.0
        return gradient / len(self.labels)
```

Giải thích `SoftmaxCrossEntropy`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [16]:
```python
class NumpyAdam:
    def __init__(
        self,
        learning_rate=1e-3,
        weight_decay=1e-4,
        beta1=0.9,
        beta2=0.999,
        epsilon=1e-8,
        gradient_clip=5.0,
    ):
        self.learning_rate = learning_rate
        self.weight_decay = weight_decay
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.gradient_clip = gradient_clip
        self.first_moment = {}
        self.second_moment = {}
        self.step_count = 0

    def step(self, named_parameters, learning_rate=None):
        self.step_count += 1
```

```
        lr = self.learning_rate if learning_rate is None else learning_rate
        for name, parameter, gradient, apply_decay in named_parameters:
            gradient = np.clip(
                gradient,
                -self.gradient_clip,
                self.gradient_clip,
            )
            if apply_decay:
                gradient = gradient + self.weight_decay * parameter

            if name not in self.first_moment:
                self.first_moment[name] = np.zeros_like(parameter)
                self.second_moment[name] = np.zeros_like(parameter)
            first = self.first_moment[name]
            second = self.second_moment[name]
            first *= self.beta1
            first += (1.0 - self.beta1) * gradient
            second *= self.beta2
            second += (1.0 - self.beta2) * gradient * gradient

            first_hat = first / (1.0 - self.beta1 ** self.step_count)
            second_hat = second / (1.0 - self.beta2 ** self.step_count)
            parameter -= lr * first_hat / (
                np.sqrt(second_hat) + self.epsilon
            )
```

Giải thích `NumpyAdam`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [17]:
```
class NumpyCNN:
    def __init__(
        self,
        input_shape,
        conv_filters=(8, 16),
        hidden_units=64,
        num_classes=2,
        random_state=42,
    ):
        height, width, channels = input_shape
        filters_1, filters_2 = conv_filters
        self.conv1 = Conv2D(
            channels,
            filters_1,
            random_state=random_state,
        )
        self.relu1 = ReLU()
        self.pool1 = MaxPool2D(2)
        self.conv2 = Conv2D(
            filters_1,
            filters_2,
            random_state=random_state + 1,
```

```
        )
        self.relu2 = ReLU()
        self.pool2 = MaxPool2D(2)
        self.flatten = Flatten()
        flattened_features = (
            (height // 4) * (width // 4) * filters_2
        )
        self.hidden = Linear(
            flattened_features,
            hidden_units,
            random_state=random_state + 2,
        )
        self.relu3 = ReLU()
        self.output = Linear(
            hidden_units,
            num_classes,
            random_state=random_state + 3,
        )

    def forward(self, x):
        x = self.pool1.forward(self.relu1.forward(self.conv1.forward(x)))
        x = self.pool2.forward(self.relu2.forward(self.conv2.forward(x)))
        x = self.flatten.forward(x)
        x = self.relu3.forward(self.hidden.forward(x))
        return self.output.forward(x)

    def backward(self, grad_logits):
        gradient = self.output.backward(grad_logits)
        gradient = self.relu3.backward(gradient)
        gradient = self.hidden.backward(gradient)
        gradient = self.flatten.backward(gradient)
        gradient = self.pool2.backward(gradient)
        gradient = self.relu2.backward(gradient)
        gradient = self.conv2.backward(gradient)
        gradient = self.pool1.backward(gradient)
        gradient = self.relu1.backward(gradient)
        return self.conv1.backward(gradient)

    def named_parameters(self):
        return [
            ("conv1.weight", self.conv1.weight, self.conv1.grad_weight, True),
            ("conv1.bias", self.conv1.bias, self.conv1.grad_bias, False),
            ("conv2.weight", self.conv2.weight, self.conv2.grad_weight, True),
            ("conv2.bias", self.conv2.bias, self.conv2.grad_bias, False),
            ("hidden.weight", self.hidden.weight, self.hidden.grad_weight, True),
            ("hidden.bias", self.hidden.bias, self.hidden.grad_bias, False),
            ("output.weight", self.output.weight, self.output.grad_weight, True),
            ("output.bias", self.output.bias, self.output.grad_bias, False),
        ]

    def state_dict(self):
        return {
            "conv1.weight": self.conv1.weight.copy(),
            "conv1.bias": self.conv1.bias.copy(),
            "conv2.weight": self.conv2.weight.copy(),
            "conv2.bias": self.conv2.bias.copy(),
```

```
            "hidden.weight": self.hidden.weight.copy(),
            "hidden.bias": self.hidden.bias.copy(),
            "output.weight": self.output.weight.copy(),
            "output.bias": self.output.bias.copy(),
        }

    def load_state_dict(self, state):
        self.conv1.weight[...] = state["conv1.weight"]
        self.conv1.bias[...] = state["conv1.bias"]
        self.conv2.weight[...] = state["conv2.weight"]
        self.conv2.bias[...] = state["conv2.bias"]
        self.hidden.weight[...] = state["hidden.weight"]
        self.hidden.bias[...] = state["hidden.bias"]
        self.output.weight[...] = state["output.weight"]
        self.output.bias[...] = state["output.bias"]

    def parameter_count(self):
        return sum(parameter.size for parameter in self.state_dict().values())
```

Giải thích `NumpyCNN`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [18]:
```
def train_numpy(model, X, y, X_val, y_val, epochs=10, batch_size=32, lr=1e-3):
    loss_fn = SoftmaxCrossEntropy()
    optimizer = NumpyAdam(learning_rate=lr)
    history = {"train_loss": [], "train_accuracy": [], "val_loss": [], "val_accurac
    rng = np.random.default_rng(42)
    for epoch in range(epochs):
        ids = rng.permutation(len(y)); total_loss = correct = 0
        for start in range(0, len(ids), batch_size):
            batch = ids[start:start + batch_size]
            logits = model.forward(X[batch])
            loss = loss_fn.forward(logits, y[batch])
            model.backward(loss_fn.backward())
            optimizer.step(model.named_parameters())
            total_loss += loss * len(batch)
            correct += np.sum(logits.argmax(1) == y[batch])
        train_loss = total_loss / len(y)
        train_accuracy = correct / len(y)
        val_logits = model.forward(X_val)
        val_loss = loss_fn.forward(val_logits, y_val)
        val_accuracy = np.mean(val_logits.argmax(1) == y_val)
        history["train_loss"].append(train_loss)
        history["train_accuracy"].append(train_accuracy)
        history["val_loss"].append(val_loss)
        history["val_accuracy"].append(val_accuracy)
        print(f"Epoch {epoch+1}/{epochs} | train_loss={train_loss:.4f} | train_acc=
    return history
```

Vòng lặp train NumPy

Mỗi batch thực hiện đúng chuỗi `forward → loss → backward → Adam step`. Đây là phần tương đương `loss.backward()` và `optimizer.step()` của PyTorch, nhưng mọi phép tính do ta tự viết.

In [19]:
```
numpy_model = NumpyCNN(
    x_train_numpy.shape[1:],
    num_classes=2,
)

numpy_history = train_numpy(
    numpy_model,
    x_train_numpy,
    y_train_numpy,
    x_val_numpy,
    y_val_numpy,
)

test_logits = numpy_model.forward(x_test_numpy)
test_accuracy = np.mean(
    test_logits.argmax(1) == y_test_numpy
)
print("Test accuracy:", test_accuracy)
```

```
Epoch 1/10 | train_loss=0.6183 | train_acc=0.6562 | val_loss=0.5643 | val_acc=0.7120
Epoch 2/10 | train_loss=0.5436 | train_acc=0.7247 | val_loss=0.5390 | val_acc=0.7310
Epoch 3/10 | train_loss=0.5043 | train_acc=0.7546 | val_loss=0.5269 | val_acc=0.7368
Epoch 4/10 | train_loss=0.4837 | train_acc=0.7656 | val_loss=0.5114 | val_acc=0.7468
Epoch 5/10 | train_loss=0.4591 | train_acc=0.7792 | val_loss=0.5055 | val_acc=0.7460
Epoch 6/10 | train_loss=0.4366 | train_acc=0.7935 | val_loss=0.4971 | val_acc=0.7550
Epoch 7/10 | train_loss=0.4156 | train_acc=0.8073 | val_loss=0.4888 | val_acc=0.7640
Epoch 8/10 | train_loss=0.3937 | train_acc=0.8211 | val_loss=0.4944 | val_acc=0.7632
Epoch 9/10 | train_loss=0.3686 | train_acc=0.8343 | val_loss=0.5061 | val_acc=0.7638
Epoch 10/10 | train_loss=0.3462 | train_acc=0.8466 | val_loss=0.5155 | val_acc=0.760
8
Test accuracy: 0.7568
```

In [20]:
```
if "results" not in globals():
    results = []


numpy_test_accuracy = float(np.mean(test_logits.argmax(1) == y_test_numpy))
results.append({"model":"NumPy CNN", "test_accuracy":numpy_test_accuracy, "test_los
histories["NumPy CNN"] = numpy_history
print(results[-1])
```

```
{'model': 'NumPy CNN', 'test_accuracy': 0.7568, 'test_loss': None, 'parameters': 671
22}
```

4. CNN bằng Keras/TensorFlow

Mỗi hàm được đặt trong một cell riêng. `image_dataset_from_directory` tự suy ra nhãn từ tên thư mục `cats` và `dogs`.

In [21]:
```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

Giải thích cell import TensorFlow

`tensorflow` là framework; `tf.keras` là API cấp cao để xây dựng và train mạng. `layers` chứa các lớp như Conv2D, MaxPooling2D, Dropout và Dense.

In [22]:
```python
def make_keras_datasets(train_paths, val_paths, test_paths,
                        image_size=(128, 128), batch_size=32):
    def load(path):
        image = tf.io.read_file(path)
        image = tf.image.decode_image(image, channels=3, expand_animations=False)
        image = tf.image.resize(image, image_size)
        image = tf.cast(image, tf.float32)
        label = tf.cast(tf.strings.regex_full_match(path, ".*dogs.*"), tf.float32)
        return image, label

    def make_dataset(paths, shuffle=False):
        dataset = tf.data.Dataset.from_tensor_slices([str(p) for p in paths])
        dataset = dataset.map(load, num_parallel_calls=tf.data.AUTOTUNE)
        if shuffle:
            dataset = dataset.shuffle(len(paths), seed=42)
        return dataset.batch(batch_size).prefetch(tf.data.AUTOTUNE)

    return make_dataset(train_paths, True), make_dataset(val_paths), make_dataset(t
```

Giải thích hàm `make_keras_datasets`

`image_dataset_from_directory` đọc ảnh theo thư mục con và tự gán nhãn (`cats` , `dogs`). `validation_split=0.2` giữ 20% train làm validation; `seed` bảo đảm chia lặp lại giống nhau. `image_size` resize ảnh; `batch_size` số ảnh mỗi batch; `shuffle=False` cho test để thứ tự dự đoán ổn định. Hàm trả về ba `tf.data.Dataset` .

In [23]:
```python
def build_keras_cnn(input_shape=(128, 128, 3)):
    """Xây CNN Keras cho phân loại chó/mèo."""
    model = keras.Sequential([
        layers.Input(shape=input_shape),
        layers.Rescaling(1.0 / 255),
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.1),
        layers.Conv2D(32, 3, activation="relu"),
        layers.MaxPooling2D(),
        layers.Conv2D(64, 3, activation="relu"),
        layers.MaxPooling2D(),
        layers.Conv2D(128, 3, activation="relu"),
        layers.GlobalAveragePooling2D(),
        layers.Dropout(0.3),
        layers.Dense(1, activation="sigmoid"),
```

```
    ])
    return model
```

Giải thích hàm `build_keras_cnn`

`Input` khai báo shape. `Rescaling` đưa pixel [0,255] về [0,1]. RandomFlip/RandomRotation chỉ augmentation khi train. Mỗi Conv2D học filters kernel; MaxPooling giảm H,W; GlobalAveragePooling2D đổi feature map thành vector số kênh; Dropout chống overfit; Dense sigmoid trả một xác suất nhị phân.

In [24]:
```
def compile_keras_model(model):
    """Cấu hình loss, optimizer và metric cho model."""
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss=keras.losses.BinaryCrossentropy(),
        metrics=[
            keras.metrics.BinaryAccuracy(name="accuracy"),
            keras.metrics.Precision(name="precision"),
            keras.metrics.Recall(name="recall"),
        ],
    )
    return model
```

Giải thích hàm `compile_keras_model`

`compile` cấu hình trước khi train. Adam cập nhật trọng số bằng gradient; learning rate `1e-3` là bước cập nhật. `binary_crossentropy` phù hợp output sigmoid và nhãn 0/1. Accuracy, Precision, Recall là các metric được tính sau mỗi batch/epoch.

In [25]:
```
# Chạy từng dòng sau khi đã kiểm tra dataset.
train_ds, val_ds, test_ds = make_keras_datasets(train_paths, val_paths, test_paths)
keras_model = compile_keras_model(build_keras_cnn())
keras_model.summary()
class TestMetricsCallback(keras.callbacks.Callback):
    def __init__(self, data):
        super().__init__()
        self.data = data
        self.test_loss = []
        self.test_accuracy = []

    def on_epoch_end(self, epoch, logs=None):
        metrics = self.model.evaluate(self.data, verbose=0, return_dict=True)
        self.test_loss.append(float(metrics["loss"]))
        self.test_accuracy.append(float(metrics["accuracy"]))

val_callback = TestMetricsCallback(val_ds)
history = keras_model.fit(
    train_ds,
    validation_data=val_ds,
    callbacks=[val_callback],
    epochs=10,
```

```
)
keras_test_metrics = keras_model.evaluate(test_ds, return_dict=True)
print(keras_test_metrics)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	
rescaling (Rescaling)	(None, 128, 128, 3)	
random_flip (RandomFlip)	(None, 128, 128, 3)	
random_rotation (RandomRotation)	(None, 128, 128, 3)	
conv2d (Conv2D)	(None, 126, 126, 32)	
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	
conv2d_1 (Conv2D)	(None, 61, 61, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	
conv2d_2 (Conv2D)	(None, 28, 28, 128)	
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	
dropout (Dropout)	(None, 128)	
dense (Dense)	(None, 1)	

Total params: 93,377 (364.75 KB)

Trainable params: 93,377 (364.75 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 130s 239ms/step - accuracy: 0.5561 - loss: 0.6808 - pre
cision: 0.5467 - recall: 0.6580 - val_accuracy: 0.6075 - val_loss: 0.6681 - val_prec
ision: 0.6045 - val_recall: 0.6184
Epoch 2/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 139s 265ms/step - accuracy: 0.6094 - loss: 0.6552 - pre
cision: 0.6168 - recall: 0.5788 - val_accuracy: 0.6273 - val_loss: 0.6426 - val_prec
ision: 0.6499 - val_recall: 0.5493
Epoch 3/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 142s 269ms/step - accuracy: 0.6396 - loss: 0.6372 - pre
cision: 0.6477 - recall: 0.6128 - val_accuracy: 0.6580 - val_loss: 0.6196 - val_prec
ision: 0.6401 - val_recall: 0.7196
Epoch 4/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 144s 273ms/step - accuracy: 0.6586 - loss: 0.6161 - pre
cision: 0.6702 - recall: 0.6250 - val_accuracy: 0.6628 - val_loss: 0.6106 - val_prec
ision: 0.6407 - val_recall: 0.7386
Epoch 5/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 204s 277ms/step - accuracy: 0.6752 - loss: 0.6026 - pre
cision: 0.6913 - recall: 0.6336 - val_accuracy: 0.6892 - val_loss: 0.5988 - val_prec
ision: 0.7071 - val_recall: 0.6445
Epoch 6/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 148s 279ms/step - accuracy: 0.6889 - loss: 0.5916 - pre
cision: 0.7094 - recall: 0.6404 - val_accuracy: 0.7000 - val_loss: 0.5766 - val_prec
ision: 0.6856 - val_recall: 0.7371
Epoch 7/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 151s 285ms/step - accuracy: 0.6937 - loss: 0.5833 - pre
cision: 0.7129 - recall: 0.6490 - val_accuracy: 0.7082 - val_loss: 0.5702 - val_prec
ision: 0.7280 - val_recall: 0.6635
Epoch 8/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 153s 288ms/step - accuracy: 0.7072 - loss: 0.5689 - pre
cision: 0.7287 - recall: 0.6606 - val_accuracy: 0.6790 - val_loss: 0.5846 - val_prec
ision: 0.8492 - val_recall: 0.4342
Epoch 9/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 154s 289ms/step - accuracy: 0.7078 - loss: 0.5654 - pre
cision: 0.7323 - recall: 0.6555 - val_accuracy: 0.7232 - val_loss: 0.5590 - val_prec
ision: 0.7179 - val_recall: 0.7341
Epoch 10/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 155s 291ms/step - accuracy: 0.7157 - loss: 0.5574 - pre
cision: 0.7397 - recall: 0.6661 - val_accuracy: 0.6858 - val_loss: 0.6053 - val_prec
ision: 0.7769 - val_recall: 0.5198
157/157 ━━━━━━━━━━━━━━━━━━━━ 12s 76ms/step - accuracy: 0.6950 - loss: 0.6009 - preci
sion: 0.7953 - recall: 0.5252
{'accuracy': 0.6949999928474426, 'loss': 0.6009196639060974, 'precision': 0.79527556
89620972, 'recall': 0.5252000093460083}
```

In [26]:

```python
if "results" not in globals():
    results = []

keras_test_accuracy = float(keras_test_metrics["accuracy"])
results.append({"model":"Keras CNN", "test_accuracy":keras_test_accuracy, "test_los
histories["Keras CNN"] = history.history
print(results[-1])
```

```
{'model': 'Keras CNN', 'test_accuracy': 0.6949999928474426, 'test_loss': 0.600919663
9060974, 'parameters': 93377}
```

Lưu kết quả Keras

Lưu accuracy/loss từ `evaluate` và số tham số bằng `count_params()`.

5. CNN bằng PyTorch

PyTorch dùng transform để xử lý ảnh, Dataset để lấy từng mẫu, DataLoader để gom batch, rồi model thực hiện forward/backward.

In [27]:
```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

Import

`transforms` resize/augmentation ảnh; `ImageFolder` đọc các thư mục con `cats` và `dogs`; `DataLoader` gom nhiều ảnh thành batch; `nn` chứa lớp mạng và loss.

In [28]:
```
class DogsCatsDataset(torch.utils.data.Dataset):
    def __init__(self, paths, transform):
        self.paths = list(paths)
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, index):
        path = self.paths[index]
        image = Image.open(path).convert("RGB")
        image = self.transform(image)
        label = 1 if path.parent.name.lower() == "dogs" else 0
        return image, label

def create_dataloaders(train_paths, val_paths, test_paths,
                       image_size=(128, 128), batch_size=32):
    train_transform = transforms.Compose([
        transforms.Resize(image_size),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
    ])
    eval_transform = transforms.Compose([
        transforms.Resize(image_size),
        transforms.ToTensor(),
    ])
    train_loader = DataLoader(
        DogsCatsDataset(train_paths, train_transform),
        batch_size=batch_size, shuffle=True,
    )
    val_loader = DataLoader(
```

```
        DogsCatsDataset(val_paths, eval_transform),
        batch_size=batch_size, shuffle=False,
    )
    test_loader = DataLoader(
        DogsCatsDataset(test_paths, eval_transform),
        batch_size=batch_size, shuffle=False,
    )
    return train_loader, val_loader, test_loader
```

`create_dataloaders`

`ImageFolder` yêu cầu cấu trúc `train/cats`, `train/dogs`. Mỗi lần truy cập dataset, transform được áp dụng; train có flip ngẫu nhiên, test không augmentation. `ToTensor` đổi `(H,W,C)` thành `(C,H,W)`. DataLoader trả batch dạng `(batch,3,128,128)` và nhãn `(batch,)`. `shuffle=True` chỉ dùng train.

In [29]:
```
def build_model():
    return nn.Sequential(
        nn.Conv2d(3, 32, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Conv2d(32, 64, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Conv2d(64, 128, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.AdaptiveAvgPool2d((1, 1)),
        nn.Flatten(),
        nn.Dropout(0.3),
        nn.Linear(128, 2),
    )
```

`build_model` và forward

`Conv2d` học kernel; `ReLU` tạo phi tuyến; `MaxPool2d` giảm H,W; `AdaptiveAvgPool2d` rút mỗi kênh còn 1 số; `Flatten` tạo vector 128; `Linear(128,2)` tạo 2 logits. Khi gọi `model(images)`, `Sequential` tự chạy forward qua từng lớp.

```
Feature map sau Conv:              (batch, 128, 32, 32)
AdaptiveAvgPool2d((1,1)):           (batch, 128, 1, 1)
Flatten():                          (batch, 128)
Linear(128, 2):                     (batch, 2)
```

`AdaptiveAvgPool2d((1,1))` lấy trung bình toàn bộ vùng `32×32` của từng kênh. Có 128 kênh nên kết quả là 128 ô `1×1`; tensor vẫn có 4 chiều. Ví dụ một kênh `2×2`:

```
⎡ 1  2 ⎤  →  (1+2+3+4)/4 = 2.5  →  ⎡2.5⎤
⎣ 3  4 ⎦                            ⎣   ⎦
```

`Flatten()` không tính trung bình và không học gì; nó chỉ bỏ hai chiều không gian `1×1` để biến `(batch,128,1,1)` thành `(batch,128)`. `nn.Linear(128,2)` nhận vector cuối

cùng và tạo 2 logits. Có thể thay bằng `torch.flatten(x, 1)` sau pooling, nhưng không nên bỏ bước đổi dạng này trước Linear.

In [30]:
```python
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0.0
    correct = total = 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        logits = model(images)          # forward
        loss = criterion(logits, labels)
        loss.backward()                 # backward
        optimizer.step()                # update weights
        total_loss += loss.item() * labels.size(0)
        correct += (logits.argmax(1) == labels).sum().item()
        total += labels.size(0)
    return total_loss / total, correct / total
```

`train_one_epoch`

`zero_grad` xóa gradient cũ; forward tạo logits; loss so sánh logits với nhãn; `backward()` dùng autograd tính gradient; `step()` cập nhật kernel/bias. `model.train()` bật Dropout train. Hàm trả loss và accuracy trung bình.

In [31]:
```python
def evaluate(model, loader, device):
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for images, labels in loader:
            logits = model(images.to(device))
            correct += (logits.argmax(1) == labels.to(device)).sum().item()
            total += labels.size(0)
    return correct / total
```

`evaluate`

`eval()` tắt Dropout; `no_grad()` không lưu đồ thị gradient. Hàm chỉ forward trên test và tính tỷ lệ dự đoán đúng.

In [32]:
```python
train_loader, val_loader, test_loader = create_dataloaders(train_paths, val_paths,
classes = ["cats", "dogs"]
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = build_model().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
print("Classes:", classes, "| Device:", device)
```

```
Classes: ['cats', 'dogs'] | Device: cpu
```

Khởi tạo train

`CrossEntropyLoss` phù hợp với 2 logits; Adam học mọi tham số model. `classes` xác nhận thứ tự nhãn (`0` / `1`). `device` chọn GPU nếu có.

In [33]:
```
torch_history = {"train_loss": [], "train_accuracy": [], "val_loss": [], "val_accur
for epoch in range(5):
    loss, accuracy = train_one_epoch(model, train_loader, optimizer, criterion, dev
    torch_history["train_loss"].append(loss)
    torch_history["train_accuracy"].append(accuracy)
    torch_history["val_accuracy"].append(evaluate(model, val_loader, device))
    test_loss = 0.0
    test_count = 0
    model.eval()
    with torch.no_grad():
        for test_images, test_labels in val_loader:
            test_logits = model(test_images.to(device))
            test_loss += criterion(test_logits, test_labels.to(device)).item() * te
            test_count += test_labels.size(0)
    torch_history["val_loss"].append(test_loss / test_count)
    print(f"Epoch {epoch + 1}/5 - loss={loss:.4f} - accuracy={accuracy:.4f}")
```

```
Epoch 1/5 - loss=0.6817 - accuracy=0.5586
Epoch 2/5 - loss=0.6602 - accuracy=0.6018
Epoch 3/5 - loss=0.6359 - accuracy=0.6411
Epoch 4/5 - loss=0.6114 - accuracy=0.6712
Epoch 5/5 - loss=0.5861 - accuracy=0.6907
```

Vòng lặp train

Mỗi epoch đi qua toàn bộ train DataLoader; mỗi batch thực hiện forward → loss → backward → optimizer.step. Nhãn test không được dùng cập nhật trọng số.

In [34]:
```
torch_test_accuracy = evaluate(model, test_loader, device)
print(f"Test accuracy: {torch_test_accuracy:.4f}")
```

```
Test accuracy: 0.7116
```

In [35]:
```
if "results" not in globals():
    results = []

torch_parameters = sum(p.numel() for p in model.parameters())
results.append({"model":"PyTorch CNN", "test_accuracy":float(torch_test_accuracy),
histories["PyTorch CNN"] = torch_history
print(results[-1])
```

```
{'model': 'PyTorch CNN', 'test_accuracy': 0.7116, 'test_loss': None, 'parameters': 9
3506}
```

Lưu kết quả PyTorch

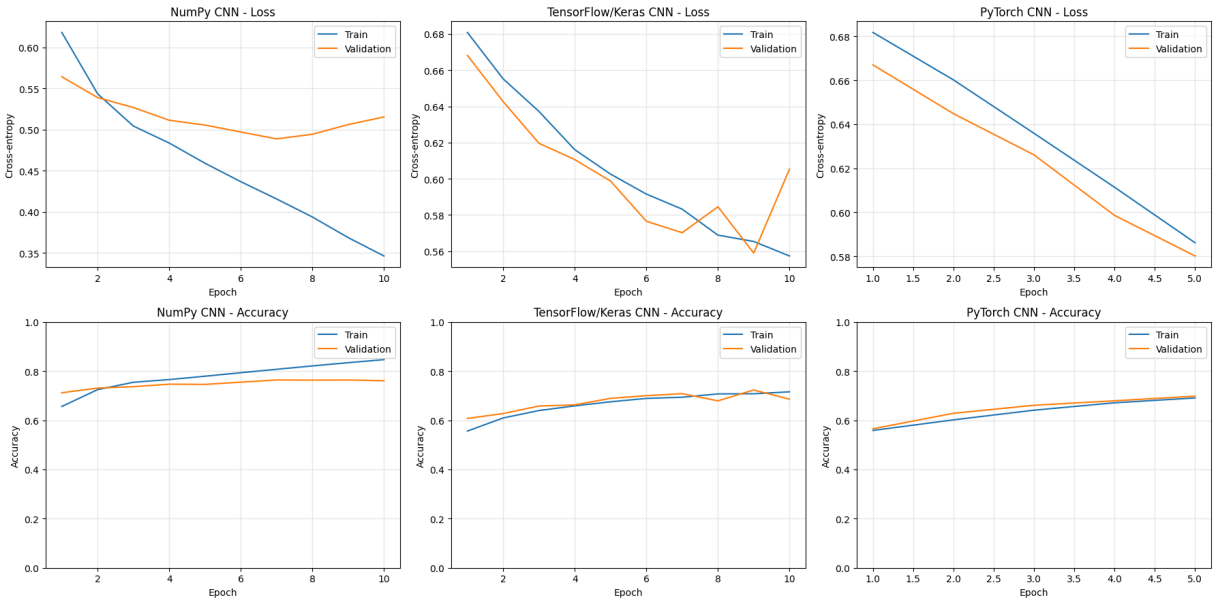
`numel()` đếm số phần tử của mọi tensor tham số.

Đánh giá cuối

Test accuracy được tính sau train trên ảnh chưa dùng tối ưu. Không gọi backward ở bước này.

Diễn biến loss và accuracy theo epoch

In [36]:
```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2, 3, figsize=(18, 9))
models = [("NumPy CNN", numpy_history, "train_loss", "val_loss", "train_accuracy",
for col, (name, values, loss_key, val_loss_key, acc_key, val_acc_key) in enumerate(
    epochs = range(1, len(values[loss_key]) + 1)
    axes[0, col].plot(epochs, values[loss_key], label="Train")
    if val_loss_key and val_loss_key in values: axes[0, col].plot(epochs, values[va
    axes[0, col].set_title(f"{name} - Loss"); axes[0, col].set_xlabel("Epoch"); axe
    epochs = range(1, len(values[acc_key]) + 1)
    axes[1, col].plot(epochs, values[acc_key], label="Train")
    if val_acc_key and val_acc_key in values: axes[1, col].plot(epochs, values[val_
    axes[1, col].set_title(f"{name} - Accuracy"); axes[1, col].set_xlabel("Epoch");
plt.tight_layout(); plt.show()
```



Nhận xét biểu đồ so sánh

- **NumPy CNN:** train loss giảm liên tục và train accuracy tăng, nhưng validation loss bắt đầu tăng từ khoảng epoch 7 trong khi validation accuracy gần như đứng yên. Đây là dấu hiệu overfitting; nên chọn checkpoint ở epoch có validation loss thấp nhất thay vì epoch cuối.
- **TensorFlow/Keras CNN:** train và validation cùng cải thiện, hai đường accuracy khá gần nhau nên chưa có overfitting mạnh. Validation loss dao động ở các epoch cuối, vì vậy cần chọn best epoch theo validation loss.

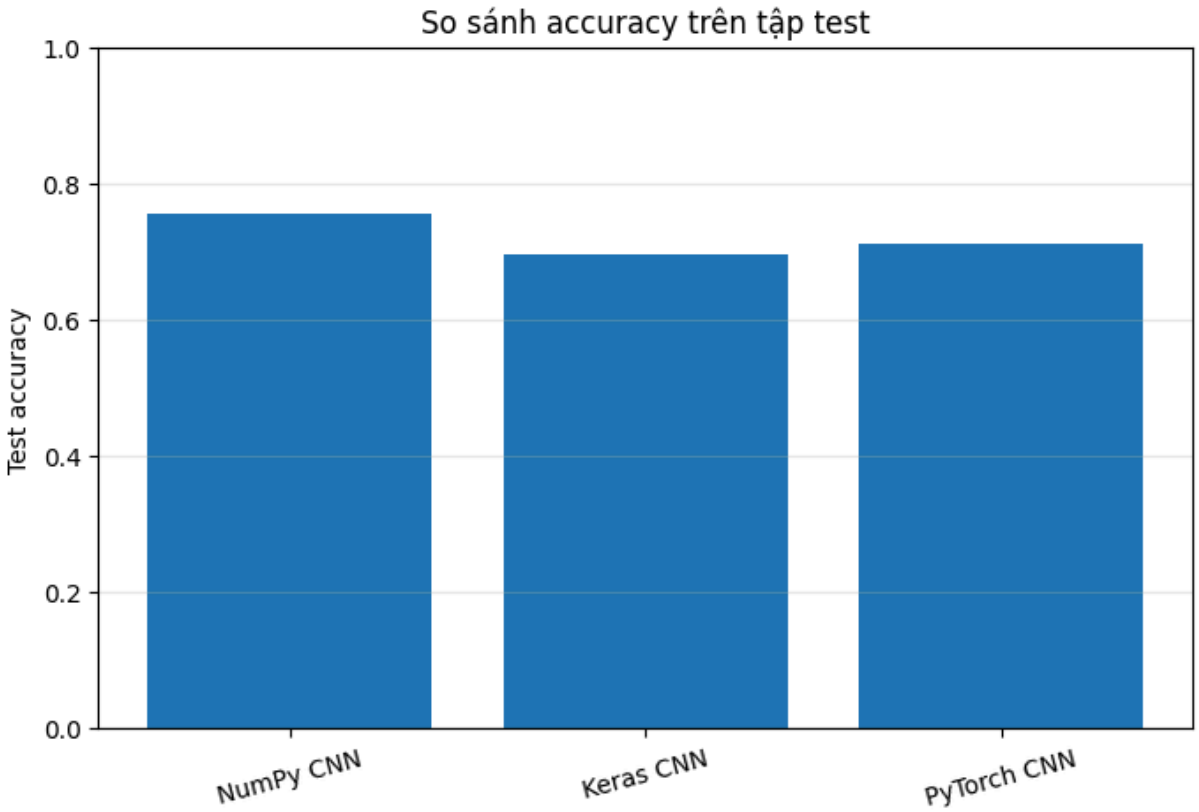
- **PyTorch CNN:** cả train và validation loss đều giảm, accuracy cùng tăng và chưa tách xa. Mô hình mới chạy ít epoch hơn nên có thể chưa học đủ (underfitting nhẹ); cần train thêm và tiếp tục theo dõi validation.

Nhìn chung, NumPy khớp train cao nhất nhưng dễ overfit; Keras ổn định hơn; PyTorch đang học ổn định nhưng chưa đạt accuracy cao. Không nên kết luận model nào tốt nhất chỉ từ hình dạng đường cong vì số epoch, kiến trúc, augmentation và hyperparameter chưa hoàn toàn giống nhau. So sánh cuối cùng nên dựa trên cùng validation protocol và test chỉ tính một lần sau khi chọn checkpoint.

So sánh ba model

In [37]:
```
import pandas as pd
import matplotlib.pyplot as plt
results_df = pd.DataFrame(results)
display(results_df)
plt.figure(figsize=(8,5))
plt.bar(results_df["model"], results_df["test_accuracy"])
plt.ylim(0,1)
plt.ylabel("Test accuracy")
plt.title("So sánh accuracy trên tập test")
plt.xticks(rotation=15)
plt.grid(axis="y", alpha=0.3)
plt.show()
```

	model	test_accuracy	test_loss	parameters
0	NumPy CNN	0.7568	NaN	67122
1	Keras CNN	0.6950	0.60092	93377
2	PyTorch CNN	0.7116	NaN	93506



Nhận xét kết quả trên tập test

Dựa vào bảng kết quả:

- **NumPy CNN** đạt `test_accuracy = 0.7568`, cao nhất trong ba mô hình. Nghĩa là mô hình dự đoán đúng khoảng **75.68%** số ảnh test. Model có `67,122` tham số, ít hơn hai model còn lại.
- **PyTorch CNN** đạt `test_accuracy = 0.7116`, đứng thứ hai. Mô hình dự đoán đúng khoảng **71.16%** ảnh test và có `93,506` tham số.
- **Keras CNN** đạt `test_accuracy = 0.6950`, thấp nhất trong ba mô hình, đúng khoảng **69.50%** ảnh test. Model có `93,377` tham số.

Xét riêng accuracy:

```
NumPy CNN    >    PyTorch CNN    >    Keras CNN
75.68%            71.16%              69.50%
```

Tuy nhiên, kiến trúc các mô hình khác nhau, hiện tại Pytorch cũng chỉ train trên 5 epoch nên chưa thể phân định được model nào là tốt nhất triệt để.

Phần B — Phân loại chữ số MNIST

MNIST – 3 CNN models

Load IDX → khám phá → chia train/validation/test → NumPy, Keras, PyTorch.

In [1]:
```
import struct
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
DATA_DIR=Path.cwd()
if not (DATA_DIR / "train-images.idx3-ubyte").exists() and (DATA_DIR / "MNIST" / "t
    DATA_DIR = DATA_DIR / "MNIST"
```

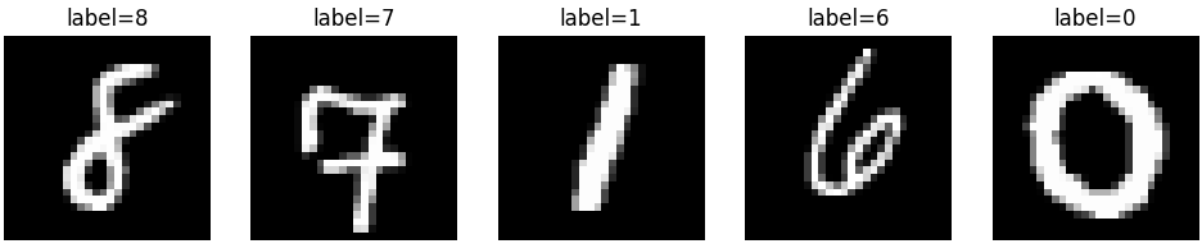
Load dữ liệu

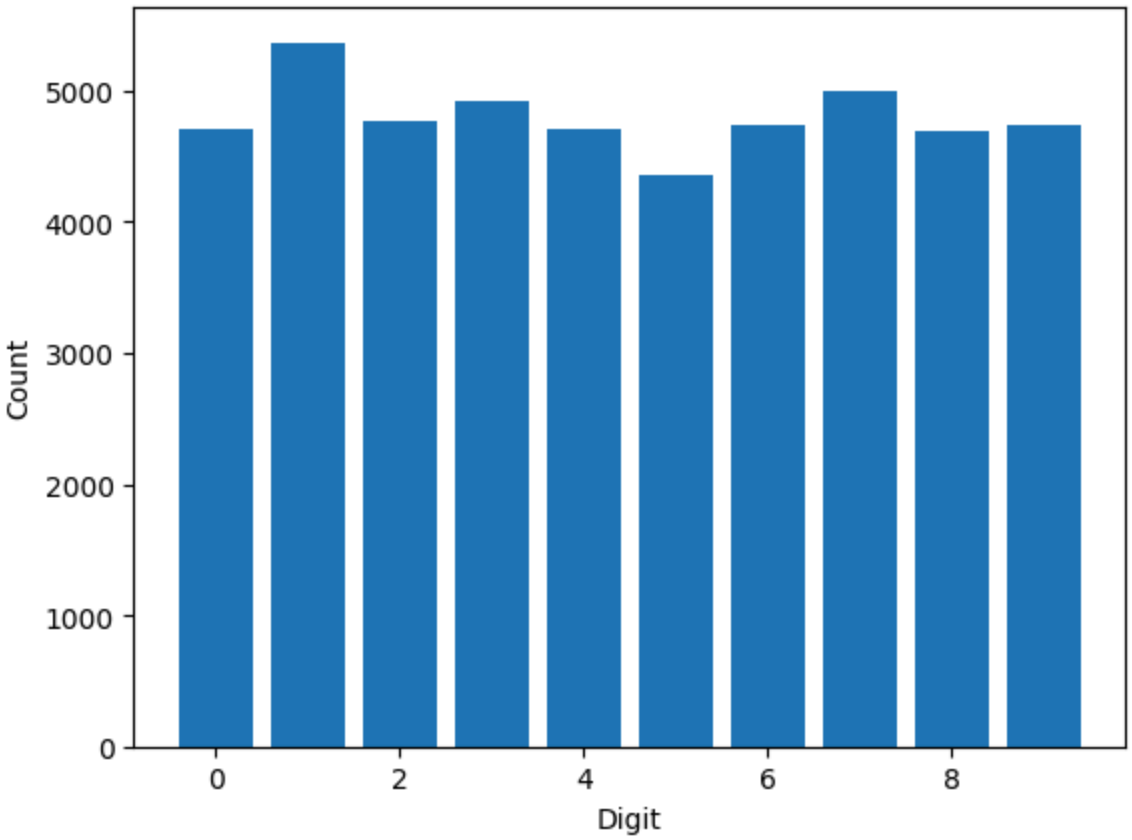
In [2]:
```
def read_images(path):
    with open(path,'rb') as f:
        _,n,h,w=struct.unpack('>IIII',f.read(16)); return np.frombuffer(f.read(),dt
def read_labels(path):
    with open(path,'rb') as f:
        _,n=struct.unpack('>II',f.read(8)); return np.frombuffer(f.read(),dtype=np.
train_images=read_images(DATA_DIR/'train-images.idx3-ubyte'); train_labels=read_lab
test_images=read_images(DATA_DIR/'t10k-images.idx3-ubyte'); test_labels=read_labels
order=np.random.default_rng(42).permutation(len(train_images)); cut=int(.8*len(orde
x_train=train_images[order[:cut],...,None].astype('float32')/255; y_train=train_lab
x_val=train_images[order[cut:],...,None].astype('float32')/255; y_val=train_labels[
x_test=test_images[...,None].astype('float32')/255; y_test=test_labels
print(x_train.shape,x_val.shape,x_test.shape)
```

```
(48000, 28, 28, 1) (12000, 28, 28, 1) (10000, 28, 28, 1)
```

Khám phá dữ liệu

In [3]:
```
fig,ax=plt.subplots(1,5,figsize=(12,3))
for i,a in enumerate(ax): a.imshow(x_train[i,...,0],cmap='gray'); a.set_title(f'lab
plt.show()
plt.bar(np.arange(10),np.bincount(y_train,minlength=10)); plt.xlabel('Digit'); plt.
```





3. CNN từ scratch bằng NumPy

Kkiến trúc hai convolution block: `Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Dense → ReLU → Dense → Softmax`. Tự viết forward/backward và Adam, không dùng autograd.

In [4]:
```python
class Conv2D:
    def __init__(
        self,
        in_channels,
        out_channels,
        kernel_size=3,
        padding=1,
        random_state=42,
    ):
        self.kernel_size = kernel_size
        self.padding = padding
        rng = np.random.default_rng(random_state)
        fan_in = kernel_size * kernel_size * in_channels
        self.weight = rng.normal(
            0.0,
            np.sqrt(2.0 / fan_in),
            size=(kernel_size, kernel_size, in_channels, out_channels),
        ).astype(np.float32)
        self.bias = np.zeros(out_channels, dtype=np.float32)

    def _extract_patches(self, x):
```

```
        p = self.padding
        x_padded = np.pad(
            x,
            ((0, 0), (p, p), (p, p), (0, 0)),
            mode="constant",
        )
        windows = np.lib.stride_tricks.sliding_window_view(
            x_padded,
            (self.kernel_size, self.kernel_size),
            axis=(1, 2),
        )
        return x_padded, windows.transpose(0, 1, 2, 4, 5, 3)

    def forward(self, x):
        self.x_padded, self.patches = self._extract_patches(x)
        return (
            np.einsum(
                "nhwklc,klco->nhwo",
                self.patches,
                self.weight,
                optimize=True,
            )
            + self.bias
        )

    def backward(self, grad_output):
        self.grad_weight = np.einsum(
            "nhwklc,nhwo->klco",
            self.patches,
            grad_output,
            optimize=True,
        )
        self.grad_bias = grad_output.sum(axis=(0, 1, 2))
        grad_patches = np.einsum(
            "nhwo,klco->nhwklc",
            grad_output,
            self.weight,
            optimize=True,
        )

        grad_padded = np.zeros_like(self.x_padded)
        out_h, out_w = grad_output.shape[1:3]
        for row in range(self.kernel_size):
            for col in range(self.kernel_size):
                grad_padded[
                    :, row:row + out_h, col:col + out_w, :
                ] += grad_patches[:, :, :, row, col, :]

        p = self.padding
        return grad_padded if p == 0 else grad_padded[:, p:-p, p:-p, :]
```

In [5]:

```
class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return np.maximum(x, 0)
```

```
    def backward(self, grad_output):
        return grad_output * self.mask
```

Giải thích `ReLU`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [6]:
```
class MaxPool2D:
    def __init__(self, pool_size=2):
        self.pool_size = pool_size

    def forward(self, x):
        n, h, w, c = x.shape
        p = self.pool_size
        if h % p != 0 or w % p != 0:
            raise ValueError("Chiều cao/rộng phải chia hết cho pool_size.")
        self.input_shape = x.shape
        self.blocks = x.reshape(n, h // p, p, w // p, p, c)
        output = self.blocks.max(axis=(2, 4))
        self.max_mask = self.blocks == output[:, :, None, :, None, :]
        self.tie_count = self.max_mask.sum(axis=(2, 4), keepdims=True)
        return output

    def backward(self, grad_output):
        distributed = (
            self.max_mask
            * grad_output[:, :, None, :, None, :]
            / self.tie_count
        )
        return distributed.reshape(self.input_shape)
```

Giải thích `MaxPool2D`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [7]:
```
class Flatten:
    def forward(self, x):
        self.input_shape = x.shape
        return x.reshape(x.shape[0], -1)

    def backward(self, grad_output):
        return grad_output.reshape(self.input_shape)
```

Giải thích `Flatten`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [8]:
```
class Linear:
    def __init__(self, in_features, out_features, random_state=43):
        rng = np.random.default_rng(random_state)
        self.weight = rng.normal(
            0.0,
            np.sqrt(2.0 / in_features),
            size=(in_features, out_features),
        ).astype(np.float32)
        self.bias = np.zeros(out_features, dtype=np.float32)

    def forward(self, x):
        self.input = x
        return x @ self.weight + self.bias

    def backward(self, grad_output):
        self.grad_weight = self.input.T @ grad_output
        self.grad_bias = grad_output.sum(axis=0)
        return grad_output @ self.weight.T
```

Giải thích `Linear`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [9]:
```
class SoftmaxCrossEntropy:
    def forward(self, logits, labels):
        shifted = logits - logits.max(axis=1, keepdims=True)
        exp_values = np.exp(shifted)
        self.probabilities = exp_values / exp_values.sum(
            axis=1,
            keepdims=True,
        )
        self.labels = labels.astype(np.int64)
        correct = self.probabilities[
            np.arange(len(labels)),
            self.labels,
        ]
        return float(-np.log(np.clip(correct, 1e-9, 1.0)).mean())

    def backward(self):
        gradient = self.probabilities.copy()
        gradient[np.arange(len(self.labels)), self.labels] -= 1.0
        return gradient / len(self.labels)
```

Giải thích `SoftmaxCrossEntropy`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [10]:
```python
class NumpyAdam:
    def __init__(
        self,
        learning_rate=1e-3,
        weight_decay=1e-4,
        beta1=0.9,
        beta2=0.999,
        epsilon=1e-8,
        gradient_clip=5.0,
    ):
        self.learning_rate = learning_rate
        self.weight_decay = weight_decay
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.gradient_clip = gradient_clip
        self.first_moment = {}
        self.second_moment = {}
        self.step_count = 0

    def step(self, named_parameters, learning_rate=None):
        self.step_count += 1
        lr = self.learning_rate if learning_rate is None else learning_rate
        for name, parameter, gradient, apply_decay in named_parameters:
            gradient = np.clip(
                gradient,
                -self.gradient_clip,
                self.gradient_clip,
            )
            if apply_decay:
                gradient = gradient + self.weight_decay * parameter

            if name not in self.first_moment:
                self.first_moment[name] = np.zeros_like(parameter)
                self.second_moment[name] = np.zeros_like(parameter)
            first = self.first_moment[name]
            second = self.second_moment[name]
            first *= self.beta1
            first += (1.0 - self.beta1) * gradient
            second *= self.beta2
            second += (1.0 - self.beta2) * gradient * gradient

            first_hat = first / (1.0 - self.beta1 ** self.step_count)
            second_hat = second / (1.0 - self.beta2 ** self.step_count)
            parameter -= lr * first_hat / (
                np.sqrt(second_hat) + self.epsilon
            )
```

Giải thích `NumpyAdam`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [11]:
```
class NumpyCNN:
    def __init__(
        self,
        input_shape,
        conv_filters=(8, 16),
        hidden_units=64,
        num_classes=10,
        random_state=42,
    ):
        height, width, channels = input_shape
        filters_1, filters_2 = conv_filters
        self.conv1 = Conv2D(
            channels,
            filters_1,
            random_state=random_state,
        )
        self.relu1 = ReLU()
        self.pool1 = MaxPool2D(2)
        self.conv2 = Conv2D(
            filters_1,
            filters_2,
            random_state=random_state + 1,
        )
        self.relu2 = ReLU()
        self.pool2 = MaxPool2D(2)
        self.flatten = Flatten()
        flattened_features = (
            (height // 4) * (width // 4) * filters_2
        )
        self.hidden = Linear(
            flattened_features,
            hidden_units,
            random_state=random_state + 2,
        )
        self.relu3 = ReLU()
        self.output = Linear(
            hidden_units,
            num_classes,
            random_state=random_state + 3,
        )

    def forward(self, x):
        x = self.pool1.forward(self.relu1.forward(self.conv1.forward(x)))
        x = self.pool2.forward(self.relu2.forward(self.conv2.forward(x)))
        x = self.flatten.forward(x)
        x = self.relu3.forward(self.hidden.forward(x))
        return self.output.forward(x)

    def backward(self, grad_logits):
        gradient = self.output.backward(grad_logits)
        gradient = self.relu3.backward(gradient)
```

```
        gradient = self.hidden.backward(gradient)
        gradient = self.flatten.backward(gradient)
        gradient = self.pool2.backward(gradient)
        gradient = self.relu2.backward(gradient)
        gradient = self.conv2.backward(gradient)
        gradient = self.pool1.backward(gradient)
        gradient = self.relu1.backward(gradient)
        return self.conv1.backward(gradient)

    def named_parameters(self):
        return [
            ("conv1.weight", self.conv1.weight, self.conv1.grad_weight, True),
            ("conv1.bias", self.conv1.bias, self.conv1.grad_bias, False),
            ("conv2.weight", self.conv2.weight, self.conv2.grad_weight, True),
            ("conv2.bias", self.conv2.bias, self.conv2.grad_bias, False),
            ("hidden.weight", self.hidden.weight, self.hidden.grad_weight, True),
            ("hidden.bias", self.hidden.bias, self.hidden.grad_bias, False),
            ("output.weight", self.output.weight, self.output.grad_weight, True),
            ("output.bias", self.output.bias, self.output.grad_bias, False),
        ]

    def state_dict(self):
        return {
            "conv1.weight": self.conv1.weight.copy(),
            "conv1.bias": self.conv1.bias.copy(),
            "conv2.weight": self.conv2.weight.copy(),
            "conv2.bias": self.conv2.bias.copy(),
            "hidden.weight": self.hidden.weight.copy(),
            "hidden.bias": self.hidden.bias.copy(),
            "output.weight": self.output.weight.copy(),
            "output.bias": self.output.bias.copy(),
        }

    def load_state_dict(self, state):
        self.conv1.weight[...] = state["conv1.weight"]
        self.conv1.bias[...] = state["conv1.bias"]
        self.conv2.weight[...] = state["conv2.weight"]
        self.conv2.bias[...] = state["conv2.bias"]
        self.hidden.weight[...] = state["hidden.weight"]
        self.hidden.bias[...] = state["hidden.bias"]
        self.output.weight[...] = state["output.weight"]
        self.output.bias[...] = state["output.bias"]

    def parameter_count(self):
        return sum(parameter.size for parameter in self.state_dict().values())
```

Giải thích `NumpyCNN`

Đây là một thành phần của CNN NumPy from scratch. Cell tự lưu các giá trị cần cho backward, thực hiện phép tính forward, rồi cung cấp gradient để optimizer cập nhật trọng số. Đọc docstring và từng phương thức theo thứ tự chạy của pipeline.

In [12]:
```
def train_numpy(model, X, y, X_val, y_val, epochs=5, batch_size=32, lr=1e-3):
    loss_fn = SoftmaxCrossEntropy()
```

```
    optimizer = NumpyAdam(learning_rate=lr)
    history = {"train_loss": [], "train_accuracy": [], "val_loss": [], "val_accurac
    rng = np.random.default_rng(42)
    for epoch in range(epochs):
        ids = rng.permutation(len(y)); total_loss = correct = 0
        for start in range(0, len(ids), batch_size):
            batch = ids[start:start + batch_size]
            logits = model.forward(X[batch])
            loss = loss_fn.forward(logits, y[batch])
            model.backward(loss_fn.backward())
            optimizer.step(model.named_parameters())
            total_loss += loss * len(batch)
            correct += np.sum(logits.argmax(1) == y[batch])
        train_loss = total_loss / len(y)
        train_accuracy = correct / len(y)
        val_logits = model.forward(X_val)
        val_loss = loss_fn.forward(val_logits, y_val)
        val_accuracy = np.mean(val_logits.argmax(1) == y_val)
        history["train_loss"].append(train_loss)
        history["train_accuracy"].append(train_accuracy)
        history["val_loss"].append(val_loss)
        history["val_accuracy"].append(val_accuracy)
        print(f"Epoch {epoch+1}/{epochs} | train_loss={train_loss:.4f} | train_acc=
    return history
```

Vòng lặp train NumPy

Mỗi batch thực hiện đúng chuỗi `forward → loss → backward → Adam step`. Đây là phần tương đương `loss.backward()` và `optimizer.step()` của PyTorch, nhưng mọi phép tính do ta tự viết.

In [13]:
```
numpy_model = NumpyCNN(
    x_train.shape[1:],
    num_classes=10,
)

numpy_history = train_numpy(
    numpy_model,
    x_train,
    y_train,
    x_val,
    y_val,
)

numpy_test_logits = numpy_model.forward(x_test)
numpy_test_loss = SoftmaxCrossEntropy().forward(numpy_test_logits, y_test)
test_accuracy = np.mean(
    numpy_test_logits.argmax(1) == y_test
)
print("Test accuracy:", test_accuracy)
```

```
Epoch 1/5 | train_loss=0.1885 | train_acc=0.9438 | val_loss=0.0784 | val_acc=0.9762
Epoch 2/5 | train_loss=0.0680 | train_acc=0.9789 | val_loss=0.0688 | val_acc=0.9777
Epoch 3/5 | train_loss=0.0499 | train_acc=0.9845 | val_loss=0.0777 | val_acc=0.9752
Epoch 4/5 | train_loss=0.0397 | train_acc=0.9875 | val_loss=0.0585 | val_acc=0.9818
Epoch 5/5 | train_loss=0.0319 | train_acc=0.9897 | val_loss=0.0472 | val_acc=0.9858
Test accuracy: 0.9858
```

In [14]:
```
histories = {}
```

In [15]:
```
if "results" not in globals():
    results = []


numpy_test_accuracy = float(np.mean(numpy_test_logits.argmax(1) == y_test))
results.append({"model":"NumPy CNN", "test_accuracy":numpy_test_accuracy, "test_los
histories["NumPy CNN"] = numpy_history
print(results[-1])
```

```
{'model': 'NumPy CNN', 'test_accuracy': 0.9858, 'test_loss': 0.04163113236427307, 'p
arameters': 52138}
```

4. CNN bằng Keras/TensorFlow

4.1 Tạo Dataset từ các mảng đã chia

In [16]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

keras_train = tf.data.Dataset.from_tensor_slices((x_train, y_train)).shuffle(10000,
keras_val = tf.data.Dataset.from_tensor_slices((x_val, y_val)).batch(128)
keras_test = tf.data.Dataset.from_tensor_slices((x_test, y_test)).batch(128)
```

`tf.data.Dataset` đóng gói ảnh và nhãn thành mini-batch; train được shuffle, validation/test không shuffle.

4.2 Xây dựng kiến trúc Keras

In [17]:
```
keras_model = keras.Sequential([
    layers.Input(shape=x_train.shape[1:]),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Mỗi Conv2D học bộ lọc; MaxPooling giảm H,W; Dense cuối có 10 nút tương ứng 10 chữ số.

4.3 Compile mô hình

In [18]:
```
keras_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

`compile` chọn Adam để cập nhật trọng số, cross-entropy cho nhãn số nguyên và accuracy để theo dõi.

4.4 Train và đánh giá Keras

In [19]:
```
keras_history = keras_model.fit(
    keras_train,
    validation_data=keras_val,
    epochs=5
)
keras_test_loss, keras_test_accuracy = keras_model.evaluate(keras_test, verbose=0)
print({'test_loss': keras_test_loss, 'test_accuracy': keras_test_accuracy})
```

```
Epoch 1/5
375/375 ━━━━━━━━━━━━━━━━━━━━ 18s 42ms/step - accuracy: 0.9367 - loss: 0.2113 - val_a
ccuracy: 0.9806 - val_loss: 0.0645
Epoch 2/5
375/375 ━━━━━━━━━━━━━━━━━━━━ 15s 41ms/step - accuracy: 0.9825 - loss: 0.0579 - val_a
ccuracy: 0.9846 - val_loss: 0.0486
Epoch 3/5
375/375 ━━━━━━━━━━━━━━━━━━━━ 16s 42ms/step - accuracy: 0.9872 - loss: 0.0406 - val_a
ccuracy: 0.9835 - val_loss: 0.0517
Epoch 4/5
375/375 ━━━━━━━━━━━━━━━━━━━━ 16s 42ms/step - accuracy: 0.9905 - loss: 0.0309 - val_a
ccuracy: 0.9892 - val_loss: 0.0331
Epoch 5/5
375/375 ━━━━━━━━━━━━━━━━━━━━ 16s 42ms/step - accuracy: 0.9925 - loss: 0.0235 - val_a
ccuracy: 0.9878 - val_loss: 0.0372
{'test_loss': 0.030604321509599686, 'test_accuracy': 0.9894999861717224}
```

In [20]:
```
if 'results' not in globals():
    results = []
results.append({
    'model': 'Keras CNN',
    'test_loss': float(keras_test_loss),
    'test_accuracy': float(keras_test_accuracy),
    'parameters': int(keras_model.count_params()),
})
print(results[-1])
```

```
{'model': 'Keras CNN', 'test_loss': 0.030604321509599686, 'test_accuracy': 0.9894999
861717224, 'parameters': 421642}
```

`fit` huấn luyện qua nhiều epoch và ghi train/validation history; `evaluate` chỉ chạy một lần trên test.

5. CNN bằng PyTorch

5.1 Tensor, Transform và DataLoader

In [25]:
```
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
Xtr = torch.tensor(x_train.transpose(0,3,1,2), dtype=torch.float32)
Xva = torch.tensor(x_val.transpose(0,3,1,2), dtype=torch.float32)
Xte = torch.tensor(x_test.transpose(0,3,1,2), dtype=torch.float32)
train_loader = DataLoader(TensorDataset(Xtr, torch.tensor(y_train)), batch_size=128
val_loader = DataLoader(TensorDataset(Xva, torch.tensor(y_val)), batch_size=128)
test_loader = DataLoader(TensorDataset(Xte, torch.tensor(y_test)), batch_size=128)
```

PyTorch dùng thứ tự kênh `(N,C,H,W)`, nên cần `transpose`. `DataLoader` chia dữ liệu thành mini-batch cho vòng lặp train.

5.2 Định nghĩa kiến trúc PyTorch

In [26]:
```
torch_model = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
    nn.Flatten(),
    nn.Linear(64 * 7 * 7, 128),
    nn.ReLU(),
    nn.Linear(128, 10),
).to(device)
```

Sau hai lần pooling, ảnh 28×28 còn 7×7; 64 kênh nên Flatten tạo 64×7×7 phần tử. Dense cuối trả 10 logits.

5.3 Loss và optimizer

In [27]:
```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(torch_model.parameters(), lr=1e-3)
```

Giải thích `CrossEntropyLoss` và `Adam`

`criterion = nn.CrossEntropyLoss()` đo độ sai lệch giữa logits model dự đoán và nhãn thật. Với MNIST, mỗi ảnh có 10 logits tương ứng các lớp `0–9`; hàm này tự kết hợp Softmax và cross-entropy nên không cần gọi Softmax trước.

Nếu xác suất model dành cho lớp đúng là `p`, loss của ảnh đó là `-log(p)`. Vì vậy dự đoán đúng với xác suất cao cho loss nhỏ, còn dự đoán sai hoặc thiếu chắc chắn cho loss lớn. Nhãn truyền vào là số nguyên, ví dụ `tensor([3, 7, 1])`.

`optimizer = torch.optim.Adam(torch_model.parameters(), lr=1e-3)` yêu cầu Adam cập nhật mọi trọng số và bias của model. `lr=0.001` là learning rate, quyết định kích thước mỗi bước cập nhật: quá lớn dễ dao động, quá nhỏ học chậm.

Trong mỗi mini-batch, quy trình luôn là:

1. `optimizer.zero_grad()` xóa gradient của batch trước (PyTorch mặc định cộng dồn gradient).
2. `output = torch_model(images)` thực hiện forward, biến ảnh thành logits.
3. `loss = criterion(output, labels)` tính mức sai.
4. `loss.backward()` thực hiện backward bằng chain rule, tính gradient `∂Loss/∂Weight` cho từng tham số.
5. `optimizer.step()` dùng các gradient đó để cập nhật trọng số theo hướng làm loss giảm.

Chu trình trên lặp qua tất cả mini-batch trong mỗi epoch. Sau nhiều epoch, kernel Conv2d và trọng số Linear dần học được đặc trưng giúp phân loại ảnh chính xác hơn.

5.3 Vòng lặp train/validation và test

In [ ]:
```
torch_history = {"loss":[], "accuracy":[], "val_loss":[], "val_accuracy":[]}
for epoch in range(5):
    torch_model.train(); total_loss=correct=total=0
    for xb,yb in train_loader:
        xb,yb=xb.to(device),yb.to(device); optimizer.zero_grad()
        out=torch_model(xb); loss=criterion(out,yb); loss.backward(); optimizer.ste
        total_loss += loss.item()*len(yb); correct += (out.argmax(1)==yb).sum().ite
    torch_model.eval(); vl=vc=vt=0
    with torch.no_grad():
        for xb,yb in val_loader:
            out=torch_model(xb.to(device)); yb=yb.to(device); vl += criterion(out,y
    torch_history['loss'].append(total_loss/total); torch_history['accuracy'].appen
    torch_history['val_loss'].append(vl/vt); torch_history['val_accuracy'].append(v
    print(
        f"Epoch {epoch + 1}/10 | "
        f"train_loss={torch_history['loss'][-1]:.4f} | "
        f"train_accuracy={torch_history['accuracy'][-1]:.4f} | "
        f"val_loss={torch_history['val_loss'][-1]:.4f} | "
        f"val_accuracy={torch_history['val_accuracy'][-1]:.4f}"
    )
```

```
torch_model.eval(); tl=tc=tt=0
with torch.no_grad():
    for xb,yb in test_loader:
        out=torch_model(xb.to(device)); yb=yb.to(device); tl += criterion(out,yb).i
torch_test_loss, torch_test_accuracy = tl/tt, tc/tt
print({'test_loss': torch_test_loss, 'test_accuracy': torch_test_accuracy})
```

```
Epoch 1/10 | train_loss=0.2729 | train_accuracy=0.9174 | val_loss=0.0919 | val_accur
acy=0.9718
Epoch 2/10 | train_loss=0.0702 | train_accuracy=0.9780 | val_loss=0.0686 | val_accur
acy=0.9780
Epoch 3/10 | train_loss=0.0466 | train_accuracy=0.9858 | val_loss=0.0485 | val_accur
acy=0.9852
Epoch 4/10 | train_loss=0.0361 | train_accuracy=0.9888 | val_loss=0.0497 | val_accur
acy=0.9850
```

In [ ]:
```
if 'results' not in globals():
    results = []
results.append({
    'model': 'PyTorch CNN',
    'test_loss': float(torch_test_loss),
    'test_accuracy': float(torch_test_accuracy),
    'parameters': int(sum(p.numel() for p in torch_model.parameters())),
})
print(results[-1])
```

`model.train()` bật chế độ train; `model.eval()` tắt dropout/batchnorm. `no_grad()` tiết kiệm bộ nhớ khi validation/test.

So sánh ba model

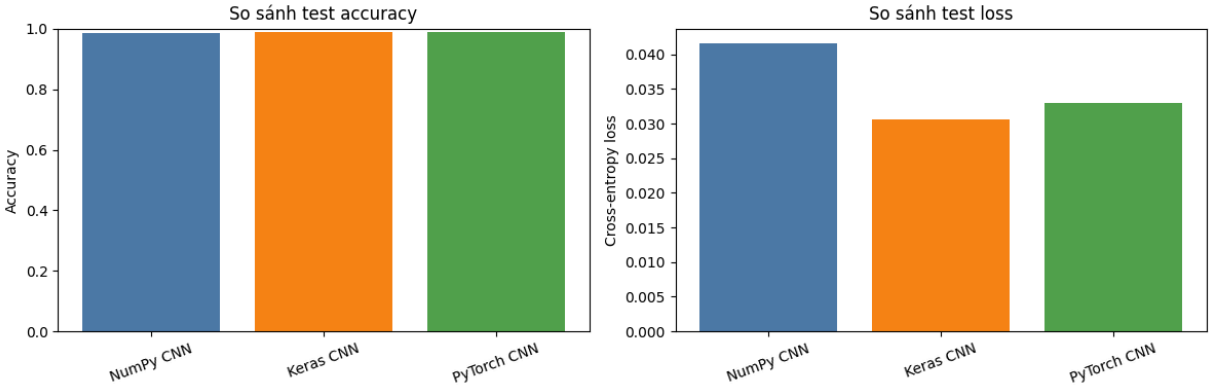
In [32]:
```
import pandas as pd
results = pd.DataFrame([
    {'model':'NumPy CNN','test_loss':numpy_test_loss,'test_accuracy':numpy_test_acc
    {'model':'Keras CNN','test_loss':keras_test_loss,'test_accuracy':keras_test_acc
    {'model':'PyTorch CNN','test_loss':torch_test_loss,'test_accuracy':torch_test_a
])
display(results)


# Biểu đồ so sánh kết quả trên tập test
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].bar(results['model'], results['test_accuracy'], color=['#4C78A8', '#F58518'
axes[0].set_title('So sánh test accuracy')
axes[0].set_ylabel('Accuracy')
axes[0].set_ylim(0, 1)
axes[0].tick_params(axis='x', rotation=20)

axes[1].bar(results['model'], results['test_loss'], color=['#4C78A8', '#F58518', '#
axes[1].set_title('So sánh test loss')
axes[1].set_ylabel('Cross-entropy loss')
axes[1].tick_params(axis='x', rotation=20)
```

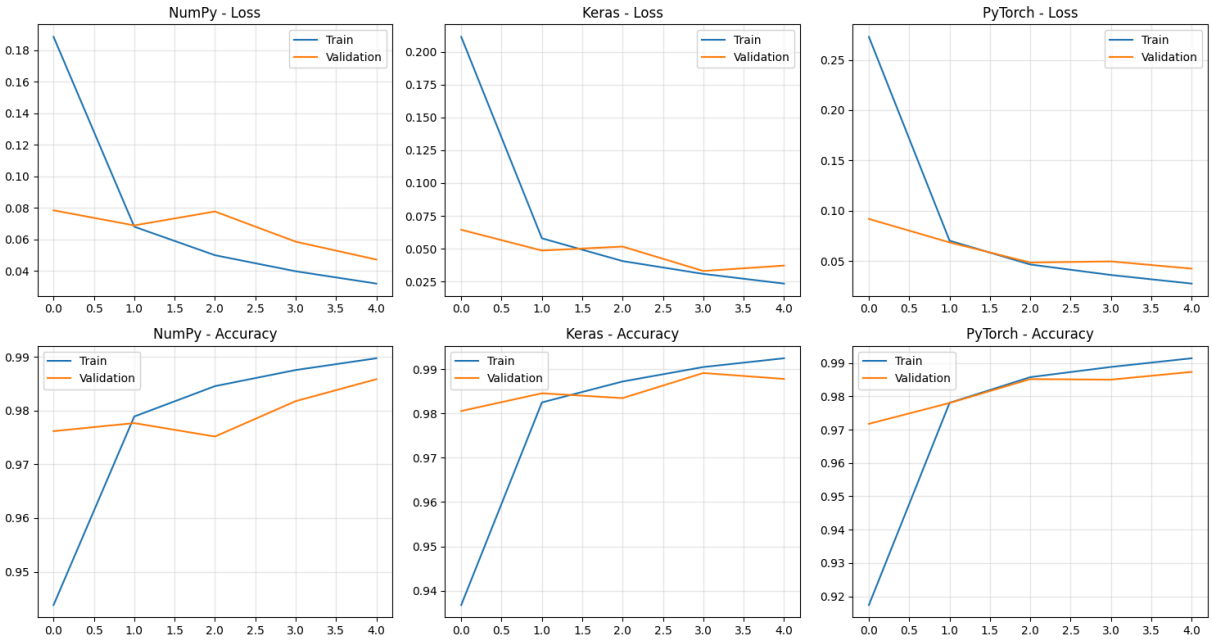
```
plt.tight_layout()
plt.show()
```

	model	test_loss	test_accuracy	parameters
0	NumPy CNN	0.041631	0.9858	52138
1	Keras CNN	0.030604	0.9895	421642
2	PyTorch CNN	0.032997	0.9883	421642



Đường cong học tập

In [33]:
```
import matplotlib.pyplot as plt
histories = [
    ('NumPy', {'loss': numpy_history['train_loss'], 'accuracy': numpy_history['trai
    ('Keras', keras_history.history),
    ('PyTorch', torch_history),
]
fig, axes = plt.subplots(2,3, figsize=(15,8))
for col,(name,h) in enumerate(histories):
    axes[0,col].plot(h['loss'], label='Train'); axes[0,col].plot(h['val_loss'], lab
    axes[1,col].plot(h['accuracy'], label='Train'); axes[1,col].plot(h['val_accurac
plt.tight_layout(); plt.show()
```



Nhận xét

Cả ba mô hình đều học tốt vì loss giảm và accuracy tăng.

Validation accuracy đôi khi cao hơn train accuracy là bình thường. Nguyên nhân có thể là dropout/augmentation chỉ được dùng khi train, hoặc tập validation tình cờ dễ hơn một chút. Điều này không tự động có nghĩa là lỗi.

Các đường cong gấp khúc lên xuống phản ánh quá trình học theo mini-batch và sự thay đổi ngẫu nhiên giữa các epoch. Dao động nhỏ là bình thường; dao động lớn hoặc validation loss tăng liên tục trong khi train loss vẫn giảm là dấu hiệu mô hình bắt đầu overfitting.

Trong biểu đồ này, các đường train và validation khá gần nhau, nên chưa có overfitting nghiêm trọng.

Vì sao cùng một CNN nhưng kết quả Dog/Cat thấp hơn MNIST?

Ảnh màu Dog/Cat là ảnh 2 chiều có **3 kênh màu (RGB)**; MNIST là ảnh 2 chiều có **1 kênh xám**.

Nguyên nhân chính nằm ở độ khó của dữ liệu:

- **MNIST đơn giản hơn:** chữ số thường nằm giữa ảnh, nền gần như đồng nhất, hình dạng ít biến đổi và mỗi lớp có đặc trưng rõ ràng.
- **Dog/Cat phức tạp hơn:** chó và mèo có nhiều giống, tư thế, kích thước, màu lông, góc chụp và phông nền khác nhau.

- **Đặc trưng bị thay đổi mạnh:** cùng là một con mèo nhưng có thể quay hướng khác, bị che khuất hoặc nằm ở vị trí khác trong ảnh.
- **Ảnh RGB có nhiều thông tin hơn:** model phải học đồng thời màu sắc, kết cấu lông, hình dạng, nền và vị trí; vì vậy số lượng tham số và dữ liệu cần thiết lớn hơn.
- **CNN đơn giản chưa đủ sâu/rộng:** kiến trúc dùng cho MNIST có thể học cạnh và nét chữ, nhưng chưa đủ khả năng ghép các đặc trưng phức tạp như tai, mắt, mõm và hình dáng con vật.
- **Dữ liệu Dog/Cat có thể ít hơn và khó cân bằng hơn**, khiến mô hình dễ overfitting hoặc tổng quát hóa kém.

Hướng cải thiện

- Tăng kích thước ảnh và số lượng dữ liệu huấn luyện.
- Dùng data augmentation như lật ngang, xoay, phóng to/thu nhỏ và thay đổi độ sáng.
- Tăng số convolution block hoặc số filters.
- Thêm Batch Normalization và Dropout phù hợp.
- Dùng learning rate nhỏ hơn hoặc Early Stopping.
- Sử dụng transfer learning với các model như MobileNet, ResNet hoặc EfficientNet đã học trên ImageNet.
- Kiểm tra và cân bằng số lượng ảnh chó/mèo.

Vì vậy, cùng một code có thể đạt accuracy rất cao trên MNIST nhưng thấp hơn đáng kể trên Dog/Cat vì độ phức tạp và mức độ biến thiên của ảnh Dog/Cat lớn hơn nhiều.